

DEAKIN UNIVERSITY

DATA CAPTURE TECHNOLOGIES

ONTRACK SUBMISSION

Use-case Design

Submitted By:

Deven GARG

s223861228

2026/08/16 01:48

Tutor:

Ashish MANCHANDA

August 15, 2026



Privacy, Literature Review and Enhanced Use Case Design

Student Name: Deven Garg

Student ID: 223861228

Task Name: 2P - Use-case Design

Contents

1. Part 1 – Privacy & Data Lifecycle Analysis	3
1.1 Original Use Case.....	3
1.2 Data Minimisation Audit.....	3
1.3 Data Retention and Lifecycle	3
1.4 Ethical Stakeholders and Consent.....	4
1.5 Privacy Notice	4
2. Part 2 – Literature Review	4
2.1 Introduction	4
2.2 Literature Review.....	4
2.3 Literature Synthesis.....	5
2.4 Research Gap.....	6
3. Part 3 – Enhanced Use Case Design	6
3.1 Refined Problem Statement.....	6
3.2 Proposed System	7
3.3 Data Capture Protocol.....	7
3.4 Data Storage and Management	8
3.5 Privacy and Ethical Considerations	8
3.6 Research Hypotheses.....	8
3.7 Validation Strategy.....	9
3.8 Data Quality and Statistical Considerations.....	9
3.9 Revised Data Lifecycle.....	10
4. Conclusion	10
5. References.....	11
6. Appendices.....	11

1. Part 1 – Privacy & Data Lifecycle Analysis

1.1 Original Use Case

The original project evaluated the performance of an **HC-SR04 ultrasonic sensor** for short-range obstacle detection. The main aim was to determine whether the sensor could provide consistent and reliable distance measurements when an object was moved to different positions. An Arduino Uno R3 was used to collect the measurements from the HC-SR04, while Python was used to receive, timestamp and store the data for later analysis.

The sensor recorded one distance measurement approximately every second for around ten minutes. Each record contained a **timestamp and distance measurement in centimetres**, which were stored locally in a CSV file called `distance_data.csv`. The final dataset contained 953 measurements. After checking the data, 952 readings were considered valid, while one reading of 802.72 cm was removed because it was outside the sensor's operating range.

The results showed that the sensor generally responded consistently when the object moved closer to or further away from it. However, the experiment was limited by the manual movement of the object, the relatively short collection period and the possibility that object angle and surrounding surfaces could affect ultrasonic measurements.

1.2 Data Minimisation Audit

The original experiment collected two main pieces of information: the **timestamp** and the **distance measured by the HC-SR04 in centimetres**. Both were relevant because the distance was needed to evaluate the sensor, while the timestamp helped show how the measurements changed over time. No personal information such as names, images, audio or location data was collected, so there was little unnecessary personal data in the dataset.

The sensor recorded a measurement every second for approximately ten minutes, resulting in 953 readings. For the original investigation, this provided enough data to identify changes and variation in the sensor readings. However, collecting a reading every second may have produced more data than necessary for a basic sensor consistency test. A sampling interval of two to five seconds could have reduced the amount of stored data while still showing the general movement pattern. The collection period could also have been extended in future testing rather than simply increasing the number of readings within a short experiment. This would provide more useful information about sensor performance over different conditions.

1.3 Data Retention and Lifecycle

The collected data was stored locally on the computer in a CSV file named `distance_data.csv`. The Python program wrote each timestamp and distance measurement directly to this file during the experiment. The data was then imported into Python for analysis and visualisation.

For the original experiment, the data was only needed for analysis and reporting. A suitable retention period for an improved system would be **30 days after the analysis is completed**. During this period, the raw data could be kept in case the results need to be checked or the analysis needs to be repeated. After 30 days, the raw measurements should be securely deleted. If longer-term information is required, only summary results such as the mean, minimum, maximum and standard deviation should be retained. This reduces unnecessary storage of raw sensor data while still keeping useful information from the experiment.

1.4 Ethical Stakeholders and Consent

In the original experiment, the main stakeholder was the student conducting the data collection because the sensor was used in a controlled environment and no personal information was collected. However, if the system were deployed in a public or shared environment, other people could be affected by the data collection. For example, occupants or visitors could be indirectly monitored through repeated distance measurements.

The HC-SR04 does not capture images, audio or personal identifiers, which reduces the privacy risk. However, repeated measurements with timestamps could potentially reveal patterns of movement if the system were used continuously. Therefore, people should be informed about the purpose of the sensor and what information is being collected.

For an improved system, a simple privacy notice should be provided before data collection. It should explain that the sensor measures distance for obstacle detection, does not intentionally collect personal information, and that the collected data will only be used for the stated purpose. Raw data should also only be accessible to authorised users.

1.5 Privacy Notice

Privacy Notice: This system uses an HC-SR04 ultrasonic sensor to measure distance for obstacle detection. The system does not intentionally collect names, images, audio, GPS information or other personal identifiers. Distance measurements are stored for analysis and system improvement only. Raw sensor data will be retained for a maximum of 30 days and then securely deleted. Access to the raw data will be limited to authorised users. Any information retained after this period will be limited to non-identifiable summary results.

2. Part 2 – Literature Review

2.1 Introduction

Ultrasonic sensing is commonly used for distance measurement and obstacle detection in robotic and embedded systems. The HC-SR04 is a low-cost sensor that measures distance by transmitting an ultrasonic signal and calculating the time taken for the reflected signal to return. The previous assessment showed that the sensor could provide generally stable measurements during a short indoor experiment, but it also identified limitations related to manual object movement, object surfaces and the short testing period.

This literature review examines existing research related to ultrasonic sensing and obstacle detection. The aim is to compare how ultrasonic sensors have been used in practical systems, identify limitations in existing approaches, and use these findings to develop a research gap for an improved version of the original HC-SR04 experiment.

2.2 Literature Review

Ultrasonic sensors are commonly used for distance measurement and obstacle detection in robotics and embedded systems. The HC-SR04 measures distance by sending an ultrasonic pulse and measuring the time taken for the echo to return. However, its measurements can be affected by factors such as distance, obstacle angle and surface characteristics.

Téllez-Garzón et al. (2025) compared ultrasonic and infrared sensors for obstacle distance measurement using different distances and detection angles. Their results showed that the HC-SR04 provided good accuracy for frontal obstacles, but accuracy decreased when obstacles

were positioned at an angle. This is relevant to the original experiment because the previous testing did not systematically control the obstacle angle.

Smaili et al. (2021) investigated distance and angle measurement errors using an HC-SR04 connected to an Arduino Uno. Their results showed that measurement accuracy varied depending on distance and highlighted the importance of comparing sensor readings with known physical measurements. This provides an improvement over the original experiment, which mainly observed changes in readings without consistently comparing them against controlled reference distances.

Haq et al. (2026) further investigated the effects of distance, angle and material on HC-SR04 measurements. Using five repetitions for each condition, they found that accuracy was generally better when the sensor was close to perpendicular to the target surface. They also observed increased measurement error at longer distances and differences between materials.

Overall, the literature shows that HC-SR04 performance depends on the conditions in which measurements are collected. **Distance, angle and surface characteristics can affect measurement accuracy and consistency.** The studies also demonstrate that repeated measurements and controlled testing conditions provide stronger evidence than manually collected single sequences. These findings support the proposed investigation and help address the limitations identified in the original experiment.

2.3 Literature Synthesis

Source	Problem Addressed	Method Used	Data Captured	Key Finding	Limitation
Téllez-Garzón et al. (2025)	Accuracy of obstacle distance measurement	HC-SR04 and infrared sensor tested at different distances and angles	Actual and measured distances	HC-SR04 performed well for frontal obstacles but accuracy decreased for angled obstacles	Mainly compared ultrasonic and infrared sensors rather than focusing only on HC-SR04
Smaili et al. (2021)	Distance and angle measurement errors	HC-SR04 with Arduino Uno; sensor readings compared with physical measurements	Distance and angle measurements	Sensor generally produced accurate measurements , but error varied with distance	Testing was relatively limited in scope
Haq et al. (2026)	Effect of angle, distance and material on HC-SR04 accuracy	Controlled tests using multiple angles, distances and materials with repeated measurements	Distance, angle, material, MAE and standard deviation	Accuracy changed with angle, distance and material; performance was strongest near perpendicular alignment	Focused on a servo-scanned configuration rather than a fixed sensor setup

The three studies provide a useful progression. Smaili et al. established that distance and angle should be considered when evaluating HC-SR04 measurements. Téllez-Garzón et al. provided a more controlled comparison of distance and angle, while Haq et al. extended the investigation by combining angle, distance and material.

2.4 Research Gap

The literature shows that the accuracy of HC-SR04 measurements is affected by **distance and obstacle orientation**, while recent research also demonstrates that **material characteristics** can influence measurement performance. However, the original assessment only investigated the general behaviour of the sensor while one object was manually moved towards and away from it. The experiment therefore showed that the sensor could respond to changes in distance, but it did not provide a controlled comparison of different testing conditions.

Although Haq et al. (2026) examined angle, distance and material together, their study used a servo-scanned configuration with specific materials and distances. A smaller fixed-sensor protocol based directly on the original Arduino Uno and HC-SR04 setup would provide a practical extension of the previous assessment. It would also allow the effects of different obstacle conditions to be evaluated using a consistent setup.

Therefore, the research gap for this assessment is:

There is a need for a controlled evaluation of how obstacle surface, angle and distance affect the consistency and reliability of HC-SR04 measurements using a fixed sensor setup.

The proposed investigation will address this gap by keeping the sensor position fixed and testing different obstacle types at controlled distances and angles. Repeated measurements will be collected for each condition and compared using the mean, range, standard deviation and invalid-reading rate. This will provide a more systematic evaluation of the sensor than the original manually controlled experiment.

3. Part 3 – Enhanced Use Case Design

3.1 Refined Problem Statement

The original project evaluated whether the HC-SR04 ultrasonic sensor could provide consistent distance measurements for short-range obstacle detection. The results showed that the sensor generally responded correctly when an object was moved closer to or further away from it. However, the original experiment only used a short testing period and manually moved an object in front of the sensor. It did not systematically investigate how different obstacle surfaces, angles and distances affect the sensor's measurements.

The refined problem is therefore:

How do obstacle surface, angle and distance affect the reliability and consistency of HC-SR04 distance measurements for short-range obstacle detection?

The improved use case will evaluate the sensor under controlled conditions using different types of obstacles and orientations. The purpose is to identify conditions that produce stable measurements and conditions that may increase measurement variation or invalid readings.

3.2 Proposed System

The enhanced system will continue to use the **HC-SR04 ultrasonic sensor and Arduino Uno**, as these were successfully used in the original experiment. The sensor will be positioned in a fixed location while different objects are placed at controlled distances from it.

Instead of testing only one manually moved object, the proposed investigation will use objects with different surface characteristics. Each object will be tested at several distances and angles. Multiple measurements will be collected for each condition so that the consistency of the sensor can be compared.

The collected measurements will be transferred to a computer and stored in a CSV file, similar to the original project. Python can then be used to calculate summary statistics and identify variation, invalid readings and differences between testing conditions.

No personal information is required for this investigation. The system only needs to record the sensor measurement, test condition and timestamp.

3.3 Data Capture Protocol

Parameter	Proposed value	Justification
Sensor	HC-SR04	Same sensor used successfully in the original experiment
Controller	Arduino Uno R3	Provides the required sensor interface and serial communication
Sampling frequency	1 reading per second	Provides regular measurements while keeping the dataset manageable
Testing duration	Multiple short test sessions	Allows different objects and conditions to be compared
Test distances	10 cm, 20 cm, 30 cm, 50 cm and 100 cm	Covers a useful range of short-range obstacle detection
Test conditions	Different surfaces and angles	Directly addresses the identified research gap
Data fields	Timestamp, distance, object type, distance setting, angle	Provides enough information to compare test conditions
Storage	Local CSV file	Simple and consistent with the original project
Personal data	None	Personal information is not required

The sensor should remain fixed during testing to reduce unnecessary variation. Each object should be tested five times under the same conditions. This will make it possible to compare the measurements and determine whether certain object characteristics produce greater variation.

3.4 Data Storage and Management

The improved system will continue using a local CSV file because this was effective in the original project. Each measurement will contain a timestamp and measured distance, along with information about the test condition. The additional fields will make it possible to determine whether measurement behaviour changes between different objects and angles.

Raw measurements will only be stored for as long as required for analysis. The proposed retention period is **30 days**, after which the raw dataset will be deleted. Summary statistics can be retained if required for reporting because they do not contain personal information.

This approach keeps the data collection simple and avoids collecting information that is not required for the research question.

3.5 Privacy and Ethical Considerations

The enhanced system is designed to minimise privacy risks by collecting only the information required to evaluate the HC-SR04 sensor. The system will record distance measurements, timestamps and test conditions such as object type, distance and angle. It will not collect names, images, audio, GPS information or other personal identifiers.

If the system is later used in a shared environment, people should be informed that an ultrasonic sensor is being used and why it is collecting measurements. Access to the raw data should also be restricted to authorised users. The proposed 30-day retention period limits the amount of time that raw measurements are stored. After this period, the raw data should be deleted, while non-identifiable summary results can be retained for reporting.

The main stakeholders are the system developer, users of the obstacle-detection system and any people who may be present in the environment where it is deployed. The system should be designed so that its safety purpose does not require unnecessary monitoring of individuals. This maintains the data-minimisation approach identified in Part 1.

3.6 Research Hypotheses

Based on the research gap identified in Part 2, the following hypotheses will be tested in the proposed investigation.

H1 – Surface:

The HC-SR04 will produce more consistent distance measurements when detecting large, flat surfaces compared with irregular or uneven surfaces.

Operationalisation:

The same objects will be measured repeatedly at controlled distances. The mean, standard deviation and range of the measurements will be compared between different surface types.

H2 – Angle:

The variation in HC-SR04 distance measurements will increase when the detected object's surface is positioned at a greater angle to the sensor.

Operationalisation:

Objects will be tested at different angles while keeping the sensor and approximate distance constant. The measurement variation and number of invalid readings will be compared between the different angles.

H3 – Distance:

Measurement variability will increase as the distance between the HC-SR04 and the obstacle increases.

Operationalisation:

Repeated measurements will be collected at several fixed distances, such as 10 cm, 20 cm, 30 cm, 50 cm and 100 cm. The standard deviation and range at each distance will be compared to determine whether measurement consistency changes with distance.

These hypotheses are directly related to the limitations of the original experiment, which identified object positioning, surface characteristics and testing conditions as factors that could influence ultrasonic measurements.

3.7 Validation Strategy

The proposed system will be validated by collecting repeated HC-SR04 measurements under controlled testing conditions. The sensor will remain fixed while obstacles are tested at the selected distances and angles. Each condition will be repeated so that the consistency of the measurements can be compared.

The proposed testing method is supported by the literature reviewed in Part 2. Previous studies found that HC-SR04 measurement performance can change with distance, angle and material characteristics. Therefore, testing these factors provides a suitable way to investigate the research gap identified in this report.

For each testing condition, the mean, minimum, maximum, range and standard deviation will be calculated. The results will be compared between different surfaces, angles and distances. A lower standard deviation and smaller range will indicate more consistent measurements.

The number of invalid readings will also be recorded for each condition. This will help identify conditions where the sensor has difficulty detecting an obstacle. Measurements will not be removed without explanation, as invalid or unusual results may provide useful information about sensor limitations.

The collected data will be sufficient to evaluate H1, H2 and H3 because each hypothesis relates directly to one of the controlled testing factors. The results will be used to determine whether the evidence supports or does not support each hypothesis.

3.8 Data Quality and Statistical Considerations

Data quality will be checked before analysing the results. The recorded measurements will first be checked for missing values, duplicate records and invalid readings. Any unusual or invalid measurements will be identified and documented rather than removed without explanation.

Repeated measurements will be collected for each testing condition to improve the reliability of the results. The **mean** will be used to describe the typical measurement, while the **standard deviation and range** will be used to measure variation between readings. The number of invalid readings will also be recorded for each condition.

The same testing procedure will be used for each surface, angle and distance so that the results can be compared fairly. Unusual values will be reviewed to determine whether they are caused by sensor limitations, testing conditions or possible recording errors.

The statistical results will be used to determine whether the HC-SR04 produces consistent measurements under different conditions. This will provide evidence for evaluating H1, H2 and H3 without making conclusions based on individual readings.

3.9 Revised Data Lifecycle

The revised data lifecycle will follow five main stages: **collection, processing, storage, analysis and deletion.**

During **collection**, the HC-SR04 will record distance measurements together with the timestamp and relevant testing conditions, including surface, angle and distance. During **processing**, the measurements will be checked for missing, duplicate and invalid values before analysis.

The processed data will be stored locally in a CSV file. During **analysis**, Python will be used to calculate the mean, range and standard deviation for each testing condition. These results will be used to evaluate the three hypotheses and compare sensor performance under different conditions.

Raw data will be retained for a maximum of **30 days** after the analysis is completed. After this period, the raw measurements will be securely deleted. If the results are required for future reporting, only non-identifiable summary statistics will be retained.

This lifecycle limits unnecessary data collection and storage while keeping the information required to evaluate the reliability and consistency of the HC-SR04 sensor.

4. Conclusion

The original assessment demonstrated that the HC-SR04 ultrasonic sensor can provide useful distance measurements for short-range obstacle detection. However, the original experiment was limited by its short testing period and manual movement of an object. The privacy and data lifecycle review also showed that the system can follow a data-minimisation approach because it does not require personal information.

The literature review showed that HC-SR04 measurement performance can be affected by factors including **obstacle surface, angle and distance**. Based on these findings, the enhanced use case was designed to investigate these factors under controlled conditions. Three hypotheses were developed to evaluate how each factor affects measurement consistency and variation.

The proposed system improves on the original experiment by using repeated measurements, controlled testing conditions and statistical measures such as mean, range and standard deviation. Invalid readings will also be recorded so that potential sensor limitations are considered rather than ignored.

The proposed data lifecycle includes controlled collection, processing, local storage, analysis and deletion of raw data after 30 days. Overall, the enhanced design provides a more systematic approach for evaluating the reliability and consistency of the HC-SR04 while maintaining appropriate privacy and data-management practices.

5. References

Haq, A. S., Firdaus, R. A., Khoiro, M., Rahmawati, E., & Winarto, N. (2026). Accuracy of HC-SR04 ultrasonic servo in servo-scanned 2D ranging. *Prisma Sains: Jurnal Pengkajian Ilmu dan Pembelajaran Matematika dan IPA*, 14(3), 1417–1431. <https://ejournal.undikma.ac.id/index.php/prismasains/article/view/19813>

Smaili, F., Beqiri, P., & Lubovci, R. (2021). Errors in distance and angle measurements of ultrasonic sensor HC-SR04. *UBT International Conference*. <https://knowledgecenter.ubt-uni.net/conference/2021UBTIC/all-events/298/?>

Téllez-Garzón, J. L., Fandiño-Pelayo, J. S., Antoine, B., & Giovanni, M. (2025). Estudio comparativo de datos empíricos obtenidos de sensores utilizados para medir la distancia de obstáculos. *Ingeniería*, 30(1), e22458. <https://revistas.udistrital.edu.co/index.php/reving/article/view/22458>

6. Appendices

Appendix A – Privacy Notice

Privacy Notice

This system uses an HC-SR04 ultrasonic sensor to measure distance for obstacle detection. The system does not intentionally collect names, images, audio, GPS information or other personal identifiers. Distance measurements are collected for sensor analysis and system improvement only. Raw sensor data will be retained for a maximum of 30 days and then securely deleted. Access to raw data will be limited to authorised users. If information is required after this period, only non-identifiable summary results will be retained.

Appendix B – HC-SR04 Ultrasonic Sensor Specifications

The HC-SR04 ultrasonic sensor was used in the original and proposed system to measure the distance between the sensor and an obstacle. The sensor operates by sending an ultrasonic signal and measuring the time taken for the echo to return. The measured time is then used to calculate the distance to the detected object.

Sensor used: HC-SR04 Ultrasonic Distance Sensor

Controller: Arduino Uno R3

Purpose: Short-range obstacle detection and distance measurement

Appendix C – Statistical Assumptions and Data Quality Checklist

The following checks will be applied to the proposed dataset before analysing the results.

Data Quality Check	Planned Approach
Missing values	Check for missing distance or timestamp values
Duplicate records	Check for repeated records
Invalid readings	Identify readings outside the expected sensor range

Outliers	Review unusually high or low measurements
Repeated measurements	Use repeated readings for each test condition
Mean	Calculate the average distance
Standard deviation	Measure variation between readings
Range	Compare minimum and maximum measurements
Consistent conditions	Keep sensor position and testing procedure consistent
Comparison	Compare results between surfaces, angles and distances

These checks will help ensure that unreliable measurements do not lead to incorrect conclusions about the performance of the HC-SR04 sensor.

Appendix D – GenAI Acknowledgement and Evidence

AI Usage Record

Generative AI was used as a learning support tool during this assessment to help understand the assessment requirements, organise the report structure, improve the clarity of written sections, review the proposed HC-SR04 investigation and identify areas that required improvement. All responses were reviewed and checked before being used. The final report, decisions about the proposed investigation, and submitted work were reviewed and completed by me.

Prompt 1 - What is the purpose of privacy and data minimisation when collecting sensor data?

Privacy and data minimisation mean that a system should only collect the information necessary for its intended purpose. For a sensor-based project such as the HC-SR04, this means focusing on the measurements needed for the experiment and avoiding unnecessary personal information. Limiting the amount of collected data can also reduce privacy risks and make the data easier to manage.

Checking Notes: -

- Compared the explanation with the privacy requirements of the assessment.
- Considered which types of data were required for the HC-SR04 experiment.
- Used this understanding when reviewing the proposed data lifecycle.

Prompt 2 - What factors can affect the accuracy and consistency of an HC-SR04 ultrasonic sensor?

The HC-SR04 measures distance using the time taken for an ultrasonic signal to travel to an object and return. Its measurements can be affected by factors such as the distance to the object, the angle of the object's surface and the

characteristics of the surface reflecting the ultrasonic signal. These factors can affect how reliably the reflected signal returns to the sensor.

Checking Notes: -

- Compared the explanation with the findings from the literature review.
- Used the information to understand why different testing conditions may produce different measurements.
- Considered these factors when developing the proposed investigation.

Prompt 3 - What makes a hypothesis suitable for an experimental sensor investigation?

A suitable experimental hypothesis makes a clear prediction about the relationship between variables that can be measured. For an HC-SR04 investigation, the independent variable could be the obstacle surface, angle or distance, while the measured distance and its variation could be used as the dependent data. A good hypothesis should therefore be specific enough to test using repeated sensor measurements.

Checking Notes: -

- Used this explanation to understand the difference between independent and dependent variables.
- Checked that the proposed hypotheses could be tested using sensor measurements.
- Confirmed that the hypotheses were connected to the research gap.

Prompt 4 - Why are repeated measurements and statistical measures useful when testing a sensor?

Repeated measurements help determine whether a sensor produces consistent results rather than relying on a single reading. The mean can be used to describe the typical measurement, while the range shows the difference between the lowest and highest readings. Standard deviation provides another measure of how much the readings vary around the mean. Checking for missing, duplicate and invalid readings also helps improve the quality of the dataset before analysis.

Checking Notes: -

- Used the explanation to understand why repeated measurements are important.
- Applied the concepts when developing the proposed data collection and validation method.
- Confirmed that mean, range and standard deviation were appropriate for describing measurement variation.

Prompt 5 - What should be considered when designing a data lifecycle for sensor measurements?

A sensor data lifecycle should consider how data is collected, processed, stored, analysed and eventually deleted. Only the data required for the purpose of the experiment should be collected. Data should be stored securely for an appropriate period and deleted when it is no longer required. For the HC-SR04 investigation, keeping only non-identifiable summary results after deleting the raw measurements can reduce unnecessary data retention.

Checking Notes: -

- Used the explanation to understand the different stages of a sensor data lifecycle.
- Applied these concepts when reviewing the proposed 30-day retention period.
- Checked that the proposed lifecycle included collection, processing, storage, analysis and deletion.

Student name: Deven Garg

Student ID: 223861228

SIT225: Data Capture Technologies

Activity 3.1: Arduino IoT Cloud and Dashboard with Arduino Nano 33 IoT devices

Arduino Cloud is your next exciting journey to build, control and monitor your connected projects. You can connect anything to Arduino Cloud including a wide range of compatible Arduino boards such as Arduino Nano 33 IoT or a third-party device that speaks Python. Arduino Cloud is an all-in-one IoT solution that empowers makers to create from anywhere, control their devices with stunning dashboards.

In this activity, you will connect your Arduino board to the cloud as a device, register a thing (in terms of a cloud variable) with your device, create a dashboard with a graphical widget which show value that is sent from the sketch running in your Arduino board.

Hardware Required

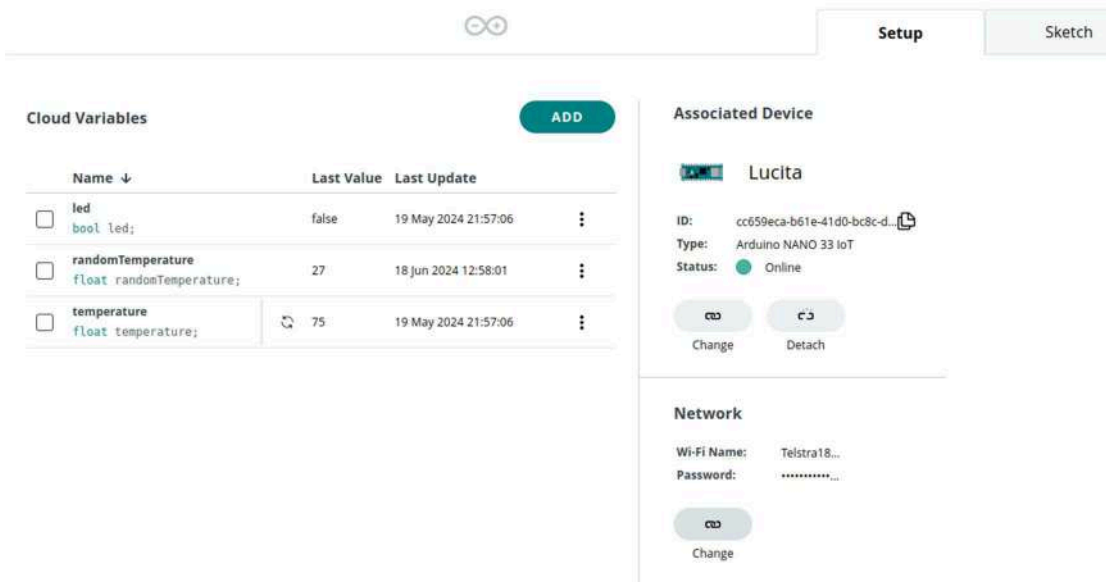
Arduino Nano 33 IoT Board
Wi-Fi hotspot (preferably your smartphone hotspot)
USB cable

Software Required

Arduino programming environment (Arduino IDE)
Arduino IoT Cloud (<https://app.arduino.cc>)

Steps

Step	Action
1	Account creation: Create an account in Arduino IoT Cloud (https://app.arduino.cc). Once done, login to your account.

	You can follow Arduino tutorial (https://support.arduino.cc/hc/en-us/articles/360016495559-Add-and-connect-a-device-to-Arduino-Cloud) for detail. This tutorial is referred to in the steps below.
2	Add the device: Follow step 1 in the tutorial to add your Arduino Nano 33 IoT device.
3	Create and configure a Thing: Follow step 2 in the tutorial to create a Thing (sensor) and attach it to the device created in step 2 above. Note that a Thing is created as a cloud variable of specific data types. In your sketch, there should exist the same variable name and type.  The screenshot shows the Arduino Cloud interface. At the top, there are tabs for 'Setup' and 'Sketch'. Below the tabs, there are two main sections. The left section is titled 'Cloud Variables' and contains a table with three variables: 'led' (bool), 'randomTemperature' (float), and 'temperature' (float). The right section is titled 'Associated Device' and shows the device 'Lucita' with its ID, type (Arduino NANO 33 IoT), and status (Online). Below this, there is a 'Network' section showing the Wi-Fi name 'Telstra18...' and a password field. Consider only a single cloud variable ‘randomTemperature’ for this activity and ignore the rest (led or temperature). To keep things simple, the “Sketch” tab in the above screenshot prepares
4	Configure your Wi-Fi: In the above screenshot, the right column shows Associated Device and Network sections. You can see “Telstra” network is shown at the time this document was prepared. This should be your smartphone hotspot which is the most convenient considering the mobility – you can carry your laptop and projects without changing in the sketch.
5	Sketch tab: There is a Sketch tab at the top right corner of the screenshot above which gives you code ready to deploy in your Arduino board.

```

13  Variables which are marked as READ/WRITE in the Cloud Thing will also have functions
14  which are called when their values are changed from the Dashboard.
15  These functions are generated with the Thing and added at the end of this sketch.
16  */
17
18  #include "thingProperties.h"
19
20  void setup() {
21    // Initialize serial and wait for port to open:
22    Serial.begin(9600);
23    // This delay gives the chance to wait for a Serial Monitor without blocking if none is found
24    delay(1500);
25
26    // Defined in thingProperties.h
27    initProperties();
28
29    // Connect to Arduino IoT Cloud
30    ArduinoCloud.begin(ArduinoIoTPreferredConnection);

```

There are 3 files in the project. The first tab shows the sketch (.ino) is generally the project name. Two other sketches are header files (.h) - thingProperties.h and arduino_secrets.h.

The thingProperties.h header looks like below.

```

1  // Code generated by Arduino IoT Cloud, DO NOT EDIT.
2
3  #include <ArduinoIoTCloud.h>
4  #include <Arduino_ConnectionHandler.h>
5
6  const char SSID[] = SECRET_SSID; // Network SSID (name)
7  const char PASS[] = SECRET_OPTIONAL_PASS; // Network password (use for WPA, or use as key for WEP)
8
9  void onRandomTemperatureChange();
10
11 float randomTemperature;
12
13 void initProperties(){
14   ArduinoCloud.addProperty(randomTemperature, READWRITE, ON_CHANGE, onRandomTemperatureChange);
15 }
16
17 WiFiConnectionHandler ArduinoIoTPreferredConnection(SSID, PASS);
18

```

It shows several items including -

- Your wifi variables (such as ssid and password which you define in arduino_secrets.h header).
- The cloud variable exactly with the same name and type you have created while creating your Thing in step 3 above.
- An initialisation function called *void initProperties() {...}* which registers the Thing variable to the cloud.
- A template function *ArduinoIoTPreferredConnection(SSID, PASS)* which works behind the scenes to connect to the Arduino Cloud using the Wi-Fi you have configured.

The arduino_secrets.h header file contains Wi-Fi ssid and password. The Secret Tab shows fields where you can input Wi-Fi information. If you want to deploy using Cloud IDE shown above (called OTA upload), it needs to upgrade your

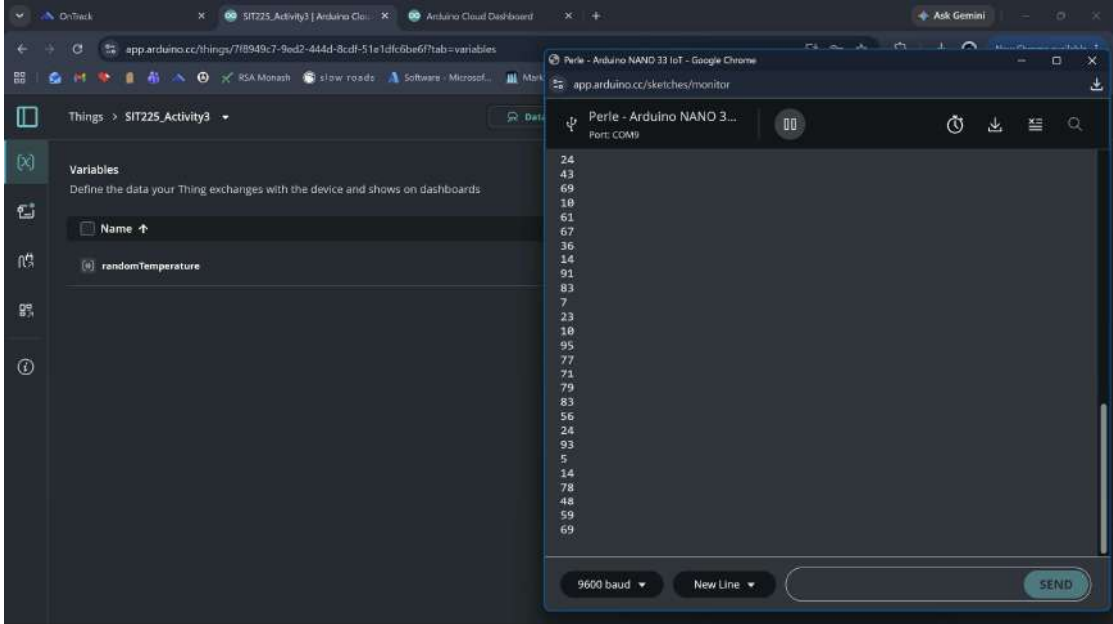
account. Instead, you can copy the sketch to your Arduino IDE installed on your computer.

```
sketch_iot_cloud.ino  arduino_secrets.h  thingProperties.h
1  #define SECRET_SSID " ";
2  #define SECRET_OPTIONAL_PASS " "
```

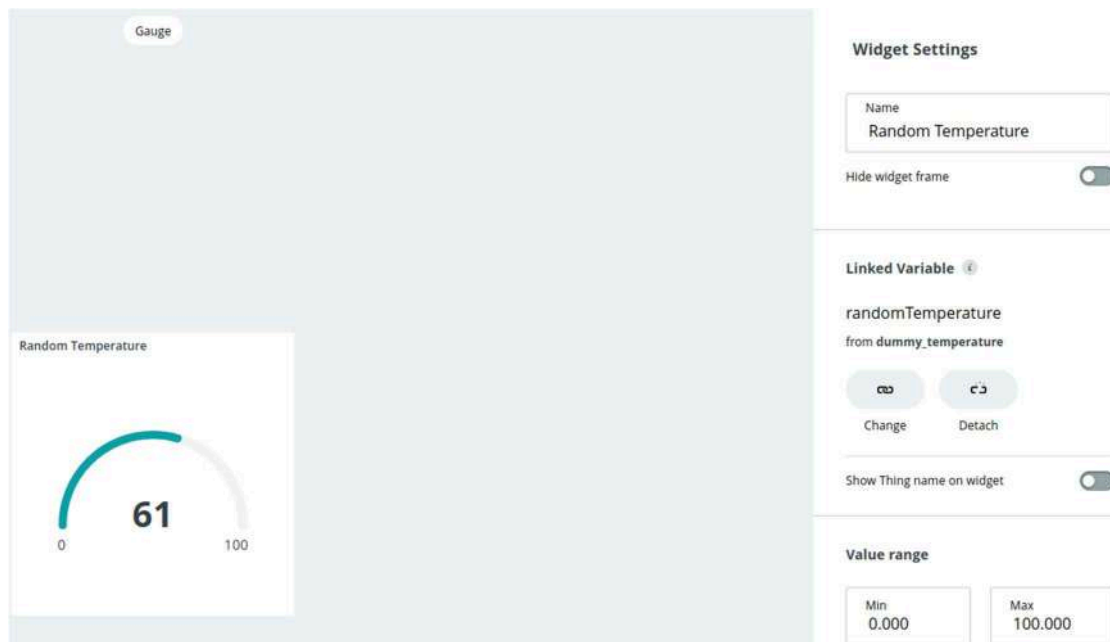
You can prepare the sketch and 2 header files as mentioned. Alternatively, you can download the code from here (https://github.com/deakin-deep-dreamer/sit225/tree/main/week_3).

The sketch in the GitHub link above shows the sketch below (*sketch_iot_cloud.ino*).

```
sketch_iot_cloud.ino  arduino_secrets.h  thingProperties.h
1  #include "thingProperties.h"
2
3  void setup() {
4      // Initialize serial and wait for port to open:
5      Serial.begin(9600);
6      // This delay gives the chance to wait for a Serial Monitor without
7      delay(1500);
8
9      // Defined in thingProperties.h
10     initProperties();
11
12     // Connect to Arduino IoT Cloud
13     ArduinoCloud.begin(ArduinoIoTPreferredConnection);
14
15     /*
16     The following function allows you to obtain more information
17     related to the state of network and IoT Cloud connection and errors
18     the higher number the more granular information you'll get.
19     The default is 0 (only errors).
20     Maximum is 4
21     */
22     setDebugMessageLevel(2);
23     ArduinoCloud.printDebugInfo();
24 }
25
26 void loop() {
27     ArduinoCloud.update();
28     // Your code here
29     randomTemperature = 1.0 * random(1, 100);
30     Serial.println("random temperature: " + String(randomTemperature));
31     delay(5*1000);
32 }
33
34 /*
35 Since Temperature is READ_WRITE variable, onRandomTemperatureChange()
36 executed every time a new value is received from IoT Cloud.
37 */
38 void onRandomTemperatureChange() {
39     // Add your code here to act upon Temperature change
40     Serial.println("--onRandomTemperatureChange");
41 }
42
```

	The loop function generates a random value between 1 and 100 and assigns to <i>randomTemperature</i> variable which is a cloud variable and write in serial port and waits for 5 seconds. Additional library to install: Arduino_CloudUtils <i>More info:</i> https://forum.arduino.cc/t/arduinoiotcloud-2-5-0-causes-compile-error/1373943/2
6	Deploy code to board: You can upload code to Arduino board from Arduino IDE. You can observe the random temperature readings in the serial monitor. Question: Screenshot the output of the serial monitor with random temperature values along with the initial Wi-Fi connection codes. Comment on the output lines. Answer:  The output shows that the Arduino connected successfully to Arduino IoT Cloud. Random temperature values are generated periodically and printed in the Serial Monitor. The values are between 1 and 100 and are sent to the cloud variable <i>randomTemperature</i>.
7	Create a dashboard: A dashboard in Arduino Cloud can be created to visualise the sensor readings sent by the Things connected to your Arduino Nano 33 IoT board. A list of Dashboard widgets is described in this tutorial (https://docs.arduino.cc/arduino-cloud/cloud-interface/dashboard-widgets).

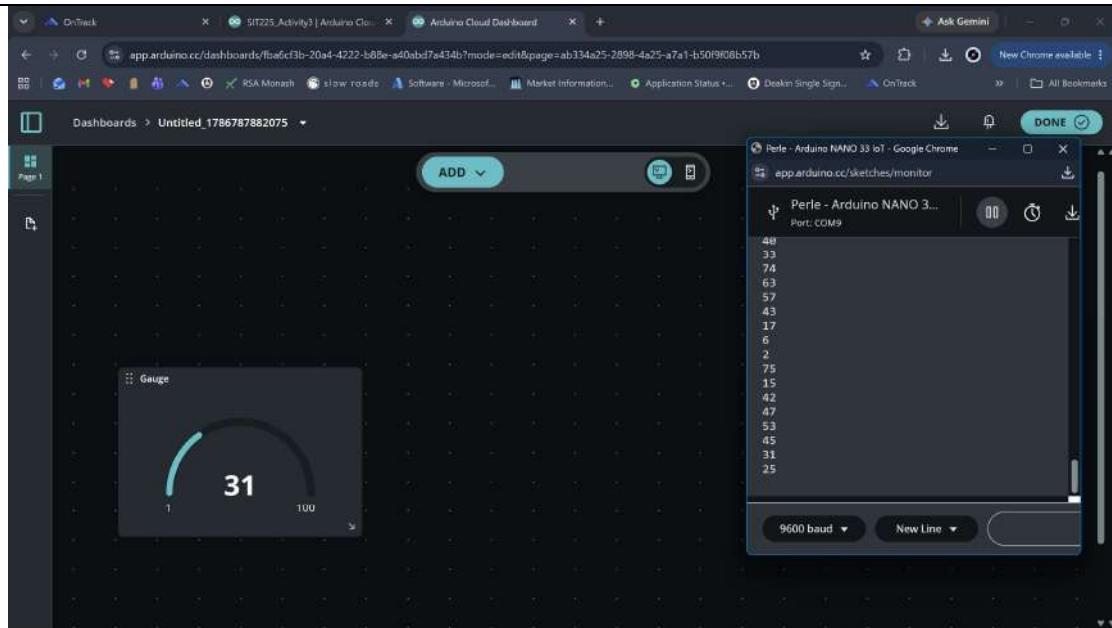
You can create a new dashboard from the Dashboards left menu items (<https://app.arduino.cc/dashboards>) where there are other menu items such as Devices and Things you have seen earlier. Creating a dashboard is simply choosing a dashboard widget, such as a Gauge and link it to the Thing cloud variable you have created.



Once you click the Done button, you should look the Gauge widget is updating based on the value shown in the Arduino IDE serial monitor.

Question: Screenshot the output of the serial monitor with random temperature values and the dashboard output so the Gauge value can be found in the serial monitor output.

Answer:



8

Question: If you recall, there is a function `initProperties()` in `thingProperties.h` file where there is a single line -

```
13 void initProperties(){
14     ArduinoCloud.addProperty(randomTemperature, READWRITE, ON_CHANGE, onRandomTemperatureChange);
15 }
```

A function `onRandomTemperatureChange()` exists (in `sketch_iot_cloud.ino` file) to respond to `ON_CHANGE` event. Can you explore what is the use of this function and when the `ON_CHANGE` event will trigger?

Answer: The `onRandomTemperatureChange()` function is used to respond when the value of the `randomTemperature` Cloud variable changes. The `ON_CHANGE` event is triggered when a new value is received or the variable is changed through Arduino IoT Cloud. This allows the Arduino to perform an action whenever the variable is updated.

Activity 3.2: Arduino IoT Cloud with custom Python devices

You can connect anything to Arduino Cloud including a wide range of compatible Arduino boards such as Arduino Nano 33 IoT or a third-party device that **speaks Python**.

In this activity, you will connect a custom Python board to Arduino IoT Cloud and synchronise to another cloud variable, the one you created earlier, *randomTemperature*. This will enable you to receive data from Arduino board in your Python script.

Hardware Required

Arduino Nano 33 IoT Board

Wi-Fi hotspot (preferably your smartphone hotspot)

USB cable

Software Required

Arduino programming environment (Arduino IDE)

Arduino IoT Cloud (<https://app.arduino.cc>)

Python 3

Steps

Step	Action
1	Thing & Device Configuration: Following the tutorial (https://docs.arduino.cc/arduino-cloud/guides/python), - Create a new Thing, by clicking on the "Create Thing" button. - Click on the "Select Device" in the "Associated Devices" section of your Thing. - Click on "Set Up New Device" and select the bottom category ("Manual Device"). Click continue in the next window and choose a name for your device. - Finally, you will see a new Device ID and a Secret Key generate. You can download them as a PDF. Make sure to save it as you cannot access your Secret Key again.
2	Create Variables: Follow the same tutorial mentioned above and do the following steps: While in Thing configuration, click on "Add Variable" which will open a

	new window. b. Name your variable temperature and select it to be of a float type. c. Just below the variable name, there is a link '<i>Sync with other Things</i>'. Click the link and it will show a list of all the variables you have created so far in your Arduino Cloud account in all the devices. d. Select 'randomTemperature' variable from Arduino board and click the button 'Synchronise Variables'. e. At this point the randomTemperature data sent from your Arduino Nano board will arrive in Arduino Cloud, then it will be written to Python device variable temperature. This value assignment triggers on_write event in the listening client and corresponding callback function is called.
3	Create Python script: Now you need to create a Python script to register to Arduino Cloud, register for a cloud variable called temperature and write a callback function for on_write event handling. You can write the code as below or download the code from here (https://github.com/deakin-deep-dreamer/sit225/blob/main/week_3/arduino_variable_sync.py).

```

1  """
2      Requirement: arduino_iot_cloud
3      Install: pip install arduino-iot-cloud
4
5      @Ahsan Habib
6      School of IT, Deakin University, Australia.
7  """
8
9  import sys
10 import traceback
11 import random
12 from arduino_iot_cloud import ArduinoCloudClient
13 import asyncio
14
15 DEVICE_ID = "bc5c0fe9-e6ef-4eb0-90de-05032ffd9a83"
16 SECRET_KEY = "3oJYfrkmSNjM4YwKGJgV0bbBn"
17
18
19 # Callback function on temperature change event.
20 #
21 def on_temperature_changed(client, value):
22     print(f"New temperature: {value}")
23
24
25 def main():
26     print("main() function")
27
28     # Instantiate Arduino cloud client
29     client = ArduinoCloudClient(
30         device_id=DEVICE_ID, username=DEVICE_ID, password=SECRET_KEY
31     )
32
33     # Register with 'temperature' cloud variable
34     # and listen on its value changes in 'on_temperature_changed'
35     # callback function.
36     #
37     client.register(
38         "temperature", value=None,
39         on_write=on_temperature_changed)
40
41     # start cloud client
42     client.start()
43
44
45 if __name__ == "__main__":
46     try:
47         main() # main function which runs in an internal infinite loop
48     except:
49         exc_type, exc_value, exc_traceback = sys.exc_info()
50         traceback.print_tb(exc_traceback, file=print)

```

You should replace DEVICE_ID and SECRET_KEY as you were given in step 1-d above.

Question: Study the code and describe in your word how the statements match to the purpose mentioned in this step-3 above?

Answer: The Python script connects to Arduino IoT Cloud using the Device ID and Secret Key. It registers the temperature cloud variable and uses a callback function to respond when the variable is updated. When a new value is received, the callback is triggered and the updated temperature value is displayed.

4

Run Python script:

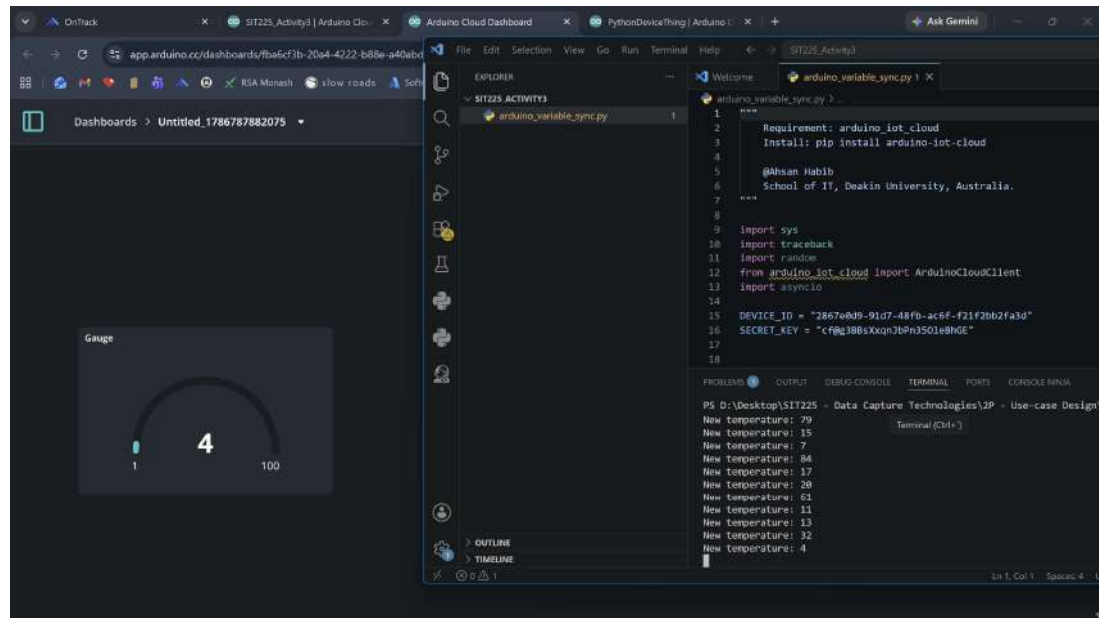
Run the python script from command line below -

```
$ python arduino_variable_sync.py
```

At this point, the command line output should show connecting to Arduino cloud and print new temperature values periodically.

Question: Screenshot the Arduino Cloud Dashboard gauge showing the Arduino side randomTemperature value and screenshot the Python command-line output showing similar values. Note that there might be some lag due to network connectivity. Explain your answer accordingly.

Answer:



The Arduino Cloud dashboard displays the randomTemperature value generated by the Arduino. The Python script receives the synchronised value through the temperature variable and displays it in the command line. A small difference in timing can occur because of network communication between the devices and Arduino Cloud.

5 **Question:** Research how you can download data from Arduino IoT Cloud, say the *randomTemperature* variable if you continue the setup running for 10 minutes or so?

Answer: Arduino IoT Cloud data can be downloaded from the cloud using the available data export options. The randomTemperature values can be collected for a selected period, such as 10 minutes, and exported for further analysis.

6 **Question:** Discuss how you can save the data while you are receiving in Python script in a file? You can discuss in a group and come up with a solution.
Hint: An algorithm of appending data can be as below. Write Python code for

the algorithm. You can put the code in the call-back function `on_temperature_changed()`.

Algorithm: Append timestamp and data value to a file:

- a. Open a file in append mode
- b. Create a CSV string `<timestamp>, <value> <NEWLINE>`
- c. Call write function and pass CSV string, otherwise, call `write_line` function and pass the CSV string removing the ending `<NEWLINE>` (otherwise, a blank new line will be written to file).
- d. Call flash to push data to be written to file immediately.
- e. Close the file.

You can modify the algorithm to make it efficient, such as instead of opening/closing the file every time, move them to the beginning and end of the execution and keeping the CSV string formation and writing and flashing to file inside the callback function.

Answer: Python can save the received values into a CSV file. Each time the `on_temperature_changed()` callback is triggered, Python can record the current timestamp and temperature value, append them to the file, and then save the changes. This creates a dataset that can later be analysed.

Student name: Deven Garg

Student ID: 223861228

SIT225: Data Capture Technologies

Activity 4.1: Collect GPS data using mobile apps

Mobile phones have significant sensing capabilities such as GPS, acceleration and lux. In this task you will use existing free mobile apps to record some GPS data.

Hardware Required

A smartphone with GPS.

Software Required

A web browser

Either Android Geo Tracker

(<https://play.google.com/store/apps/details?id=com.ilyabogdanovich.geotracker&hl=en>)

Or iOS myTracks (<https://itunes.apple.com/au/app/mytracks-the-gps-logger/id358697908?mt=8>)

You may install and try out any other app well as, as long as they can track GPS & export tracks to GPX format.

Steps:

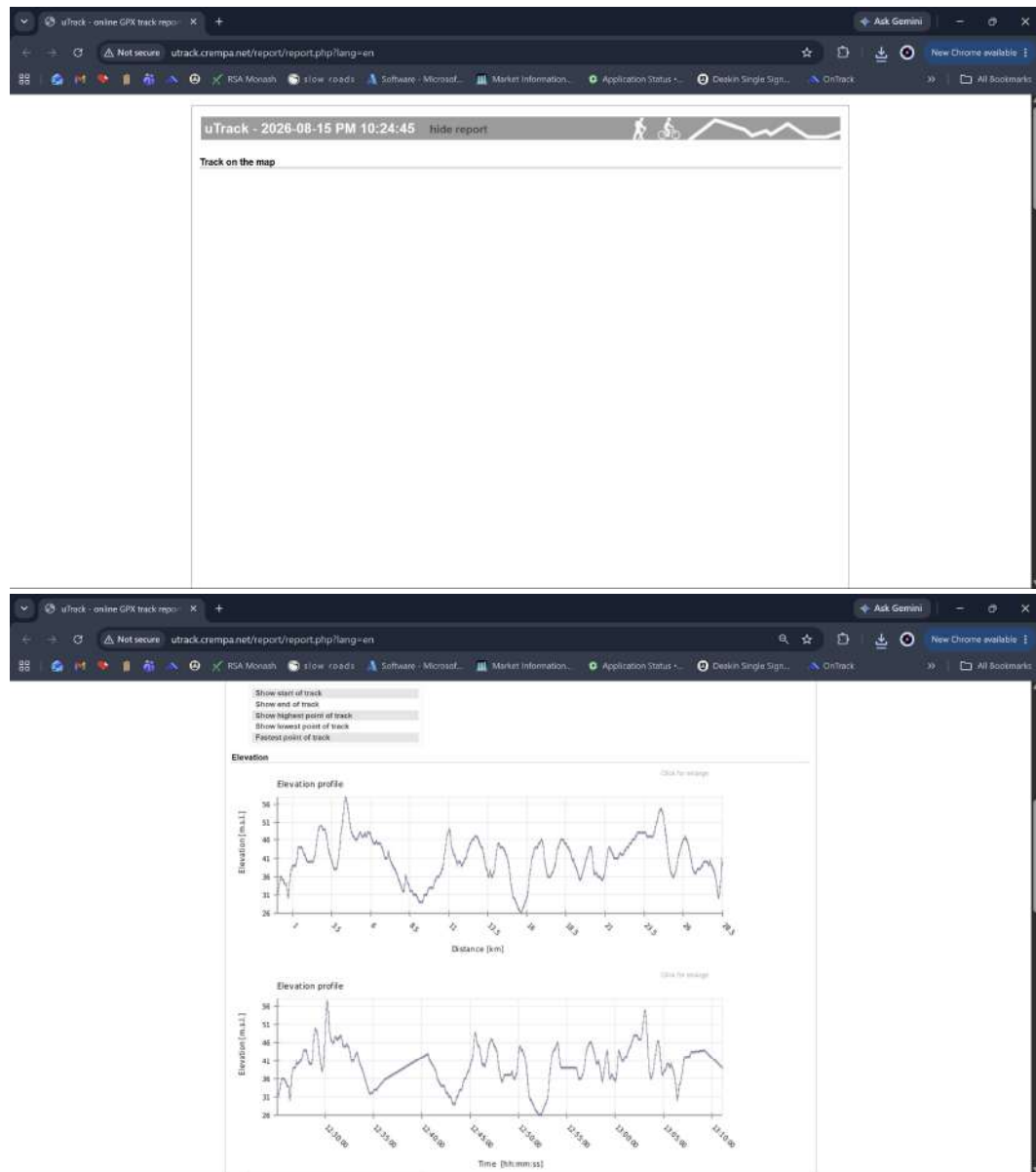
Step	Action
1	Track a journey using one of the installed mobile apps. It is best if your tracked journey spans at least 5 kilometers. Export your track to GPX format and save the file to your computer. Alternatively, you can email the .gpx file from your phone and download it to your computer Question: Upload your .gpx file to a cloud storage (such as OneDrive) and share a public link here.

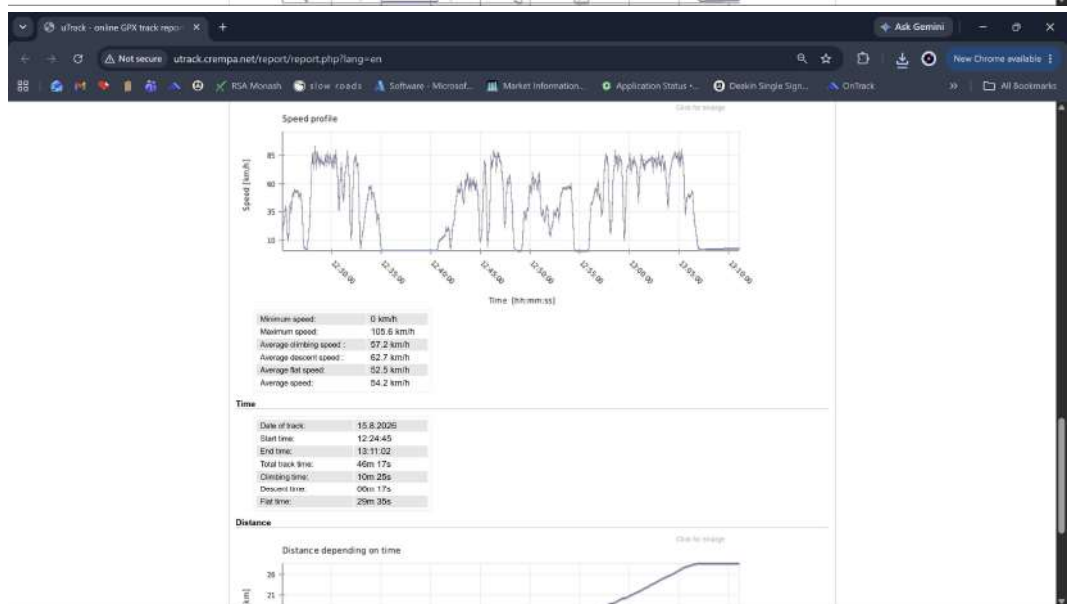
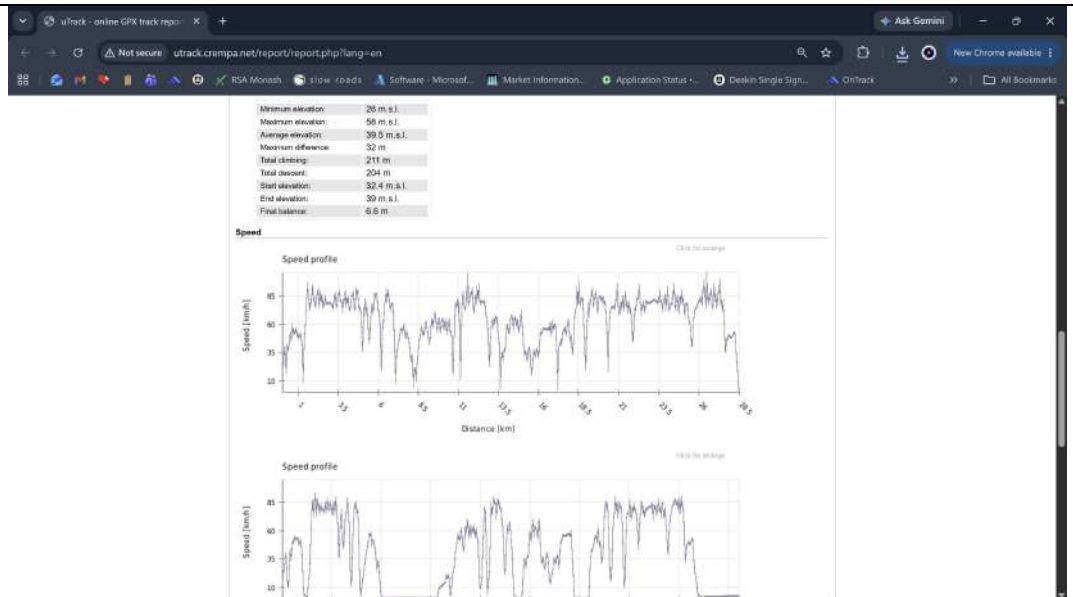
Answer: https://deakin365-my.sharepoint.com/:f:/r/personal/s223861228_deakin_edu_au/Documents/SIT225%20-%20%202P%20-%20Use%20Case%20Design?d=w020d789fad234b6698a7a09c19b9e352&csf=1&web=1&e=2usodw

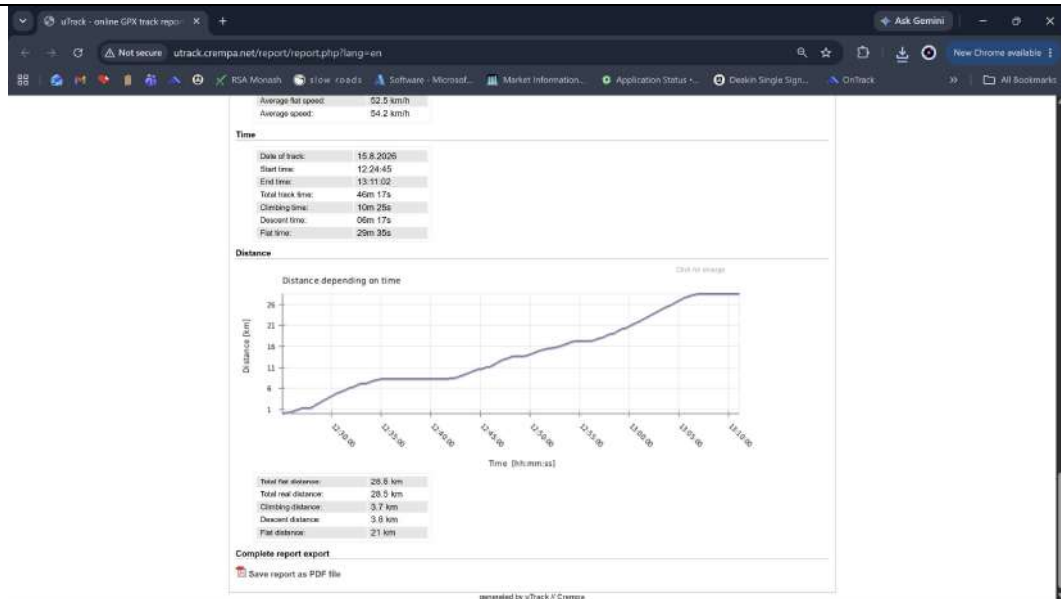
2

Question: Open a browser on your computer and go to <http://utrack.crempa.net/> . Upload your .GPX file to the site and click ‘Generate Report’. Take a screenshot and include it here.

Answer:







3

Question: What information can you see from the generated report?

Note: Depending on the app you use to record the GPS data, timestamps could be either in local time (that is AEST if you are in Melbourne) or sometimes it could be in UTC.

Answer: The generated report provides information such as the distance travelled, journey duration, speed, timestamps, GPS coordinates and the route taken. It can also show changes in speed and the geographical path of the journey. The timestamps may be shown in local time or UTC depending on the GPS app used.

4

Question: Discuss in a group of 3 or 4 students to identify potential risks (more than one risk) if your data, which was collected by a service provider such as Google Map, was misused without your concern. Also suggest remedies - what kind of policies the service providers should adopt to mitigate these risks.

Answer: Misuse of GPS data could create privacy and safety risks. For example, someone could determine where a person lives, works or regularly travels, which could lead to stalking, unwanted tracking or theft. Service providers should collect only necessary location data, clearly explain how it will be used, obtain user consent and allow users to control or delete their data. Strong security and limited access should also be used to prevent unauthorised access.

Activity 4.2: The Ethics of Data Gathering

Digital privacy is top of mind for many Australians. With weekly data breach scandals, individuals are becoming more aware and concerned about who has their data, and more importantly, who controls how that information is gathered, used and shared. In this task, you will summarise a blog post regarding what should Australian businesses know regarding the ethics of data gathering.

Steps:

Step	Action
1	Question: Study the web material which can be found here (https://www.insideinfo.com.au/business-intelligence-and-analytics/ethics-data-gathering-what-should-australian-businesses-know). Discuss data legislation in Australia and (un)ethical use of customer data. Reflect your viewpoint. Answer: Australian businesses need to follow privacy laws such as the Privacy Act 1988 and the Australian Privacy Principles. Businesses should be transparent about what personal data they collect, why they collect it and how it will be used. Ethical data gathering means collecting information that is necessary, keeping it secure and giving customers reasonable control over their information. Unethical behaviour includes collecting excessive information, using data for purposes customers did not expect and sending excessive direct marketing. In my view, businesses should only collect the data they genuinely need and should clearly explain its use to customers.

Activity 4.3: The ethical dilemma of self-driving cars

Self-driving cars are already cruising the streets today. And while these cars will ultimately be safer and cleaner than their manual counterparts, they can't completely avoid accidents altogether. In this video, you will explore and reflect on how the car should be programmed if it encounters an unavoidable accident.

Steps:

Step	Action
1	Question: Study the web material which can be found here (https://www.youtube.com/watch?v=ixloDYVfKA0). Reflect critically the ethical considerations and discuss to what extent you agree or disagree with justification. Answer: Self-driving cars create difficult ethical decisions because an accident may be unavoidable and the car may have to choose between different harmful outcomes. I believe the system should prioritise reducing the overall risk of serious injury while treating people's lives fairly. These decisions should not be based on a person's age, wealth or social status. I agree that manufacturers and governments need to establish clear ethical rules and safety standards before these decisions are left entirely to computer systems.

Activity 4.4: The ethical dilemma of abusing generative language models

The use of generative language models such as ChatGPT is increasing in our daily life. A specific question can be asked in the chat-bot command prompt which generates related answers. Students often find the chat-bot a handy tool to ask any study related questions to get a direct answer and feels reluctant to search online and compile an answer by navigating through several websites. While the use of chat-bots are useful, their improper use can limit the students ability to learn. For example, using chat-bots to generate a direct answer to parts of an assignment should be avoided considering an ethical, as well as productive point-of-view.

Steps:

Step	Action
1	Question: How you feel chat-bots (such as ChatGPT) should be used in the context of solving an assignment question? Reflect critically the ethical and learning considerations. Discuss the question in a group of 3 to 4 students and share your and some of the other students' thoughts that attracted your attention. Answer: I think chatbots such as ChatGPT should be used as a learning tool rather than simply to generate answers for assignments. They can be useful for explaining difficult concepts, giving examples, checking understanding and helping with ideas. However, submitting AI-generated answers as your own work is unethical because it reduces learning and does not accurately demonstrate your knowledge. Students should use AI responsibly, check the information it provides and follow their university's rules about AI use.

Student name: Deven Garg

Student ID: 223861228

SIT225: Data Capture Technologies

Activity 5.1: Firebase Realtime database

The Firebase Realtime Database is a cloud-hosted NoSQL database that lets you store and sync data between your users in real-time. Data is stored as JSON and synchronized in real-time to every connected client. In this activity, you will set up and perform operations such as queries and updates on the database using Python programming language.

Hardware Required

No hardware is required.

Software Required

Firebase Realtime database
Python 3

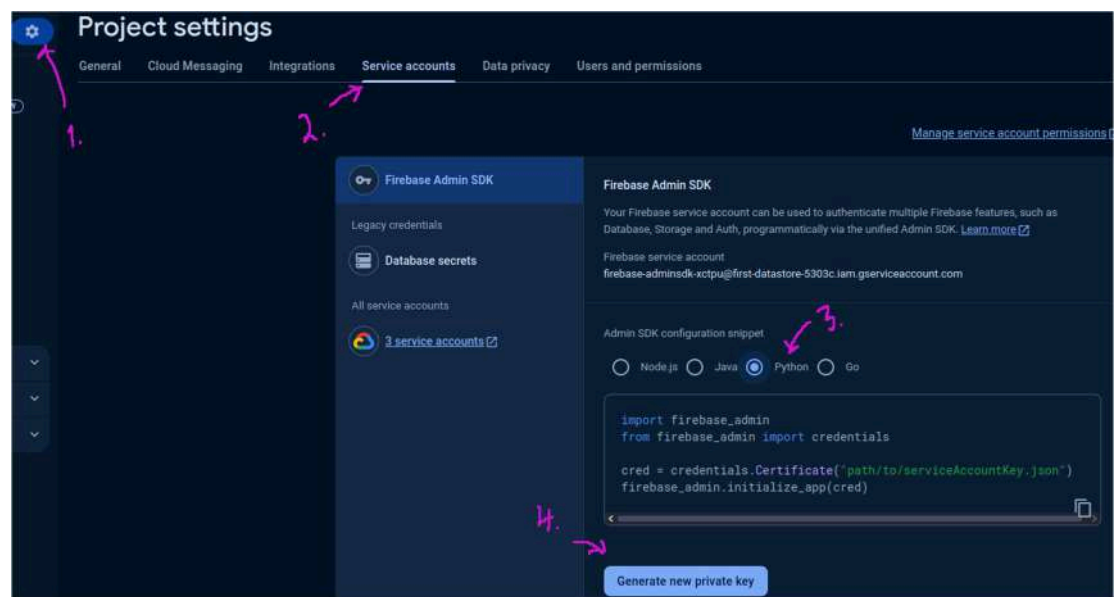
Steps

Step	Action
1	Create an Account: First, you will need to create an account in the Firebase console, follow instructions in the official Firebase document (https://firebase.google.com/docs/database/rest/start).
2	Create a Database: Follow the above Firebase document to create a database. When you click on Create Database, you have to specify the location of the database and the security rules. Two rules are available – locked mode and test mode; since we will be using the database for reading, writing, and editing, we choose test mode.
3	Setup Python library for Firebase access: We will be using Admin Database API, which is available in <i>firebase_admin</i> library. Use the below command in the command line to install. You can

follow a Firebase tutorial here (<https://www.freecodecamp.org/news/how-to-get-started-with-firebase-using-python>).

```
$ pip install firebase_admin
```

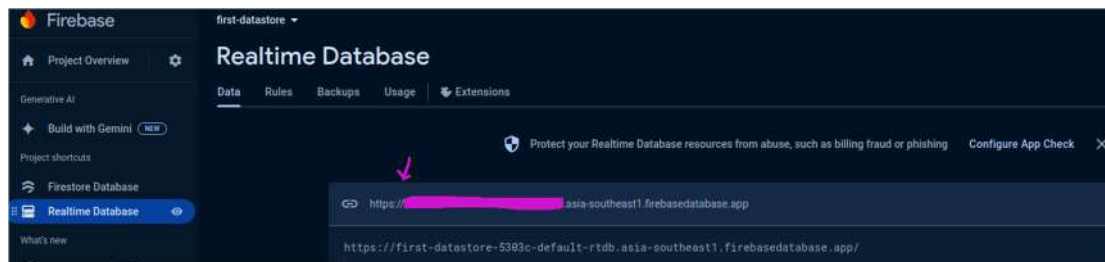
Firebase will allow access to Firebase server APIs from Google Service Accounts. To authenticate the Service Account, we require a private key in JSON format. To generate the key, go to project settings, click Generate new private key, download the file, and place it in your current folder where you will create your Python script.



- 4 **Connect to Firebase using Python version of Admin Database API:**
A credential object needs to be created to initialise the Python library which can be done using the Python code below. Python notebook can be downloaded here (https://github.com/deakin-deep-dreamer/sit225/blob/main/week_5/firebase_explore.ipynb).

```
1 import firebase_admin
2
3 databaseURL = 'https://XXX.firebaseio.com/'
4 cred_obj = firebase_admin.credentials.Certificate(
5 | 'first-datastore-5303c-firebase-adminsdk-xctpu-c9902044ac.json'
6 | )
7 default_app = firebase_admin.initialize_app(cred_obj, {
8 | 'databaseURL': databaseURL
9 | })
```

The databaseURL is a web address to reach your Firebase database that you have created in step 2. This URL can be found in the Data tab of Realtime Database.



If you compile the code snippet above, it should do with no error.

5

Write to database Using the set() Function:

We set the reference to the root of the database (or we could also set it to a key value or child key value). Data needs to be in JSON format as below.

```

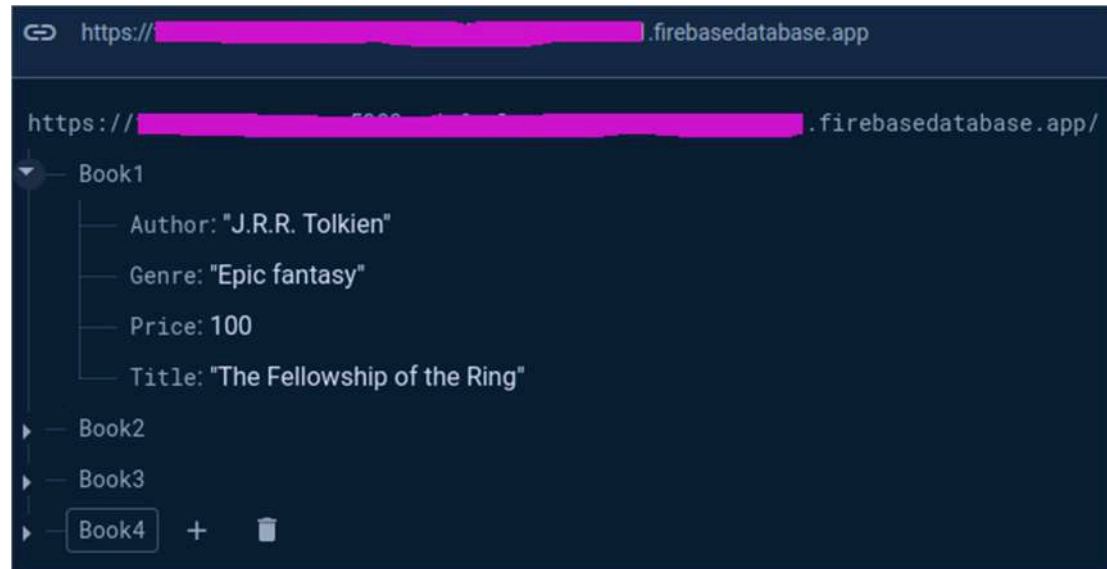
1  from firebase_admin import db
2
3  # A reference point is always needed to be set
4  # before any operation is carried out on a database.
5  #
6  ref = db.reference("/")
7
8  # JSON format data (key/value pair)
9  data = { # Outer {} contains inner data structure
10     "Book1":
11         {
12             "Title": "The Fellowship of the Ring",
13             "Author": "J.R.R. Tolkien",
14             "Genre": "Epic fantasy",
15             "Price": 100
16         },
17     "Book2":
18         {
19             "Title": "The Two Towers",
20             "Author": "J.R.R. Tolkien",
21             "Genre": "Epic fantasy",
22             "Price": 100
23         },
24     "Book3":
25         {
26             "Title": "The Return of the King",
27             "Author": "J.R.R. Tolkien",
28             "Genre": "Epic fantasy",
29             "Price": 100
30         },
31     "Book4":
32         {
33             "Title": "Brida",
34             "Author": "Paulo Coelho",
35             "Genre": "Fiction",
36             "Price": 100
37         }
38 }
39
40 # JSON format data is set (overwritten) to the reference
41 # point set at /, which is the root node.
42 #
43 ref.set(data)

```

A reference point always needed to be set where the data read/write will take place. In the code above, the reference point is set at the root of the NoSQL Document, where consider the database is a JSON tree and / is the root node

of the tree). The set() function writes (overwrites) data at the set reference point.

You can visualise the data in the Firebase console as below -



6 Read data using get() function:

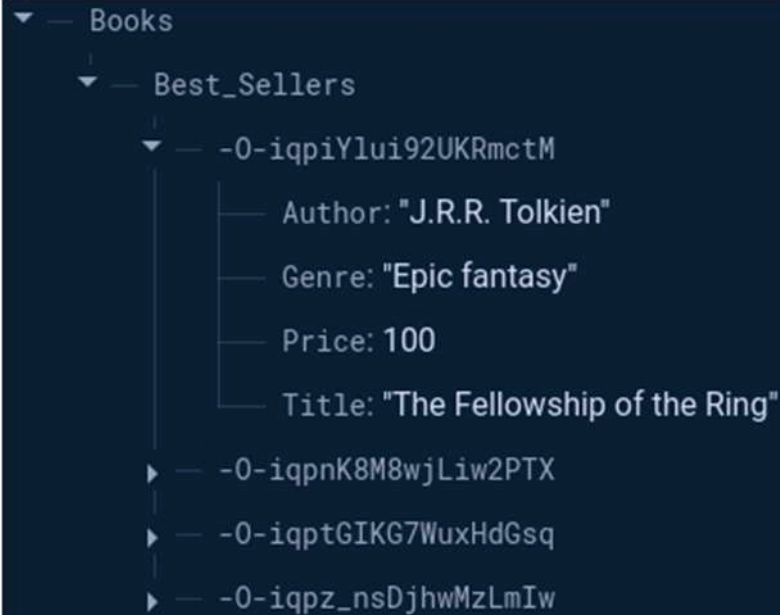
Data can be read using get() function on the reference set beforehand, as shown below.

```
1  ref = db.reference("/") # set ref point
2
3  # query all data under the ref
4  books = ref.get()
5  print(books)
6  print(type(books))
7
8  # print each item separately
9  for key, value in books.items():
10 |     print(f"{key}: {value}")
11
12
13 # Query /Book1
14 ref = db.reference("/Book1")
15 books = ref.get()
16 print(books)
```

✓ 0.3s

```
{'Book1': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}}
<class 'dict'>
Book1: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}
Book2: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}
Book3: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}
Book4: {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'}
```

	Consider the reference set in line 1 and the output compared to the reference set at line 14 and the bottom output line to understand the use of db.reference() and ref.get().
7	Write to database Using the push() Function: The push() function saves data under a <i>unique system generated key</i>. This is different than set() where you set the keys such as Book1, Book2, Book3 and Book4 under which the content (author, genre, price and title) appears. Let's try to push the same data in the root reference. Note that since we already has data under root / symbol, setting (or pushing) in the same reference point will eventually rewrite the original data. <pre>1 # Write using push() function 2 # Note that a set() is called on top of push() 3 # 4 ref = db.reference("/") 5 ref.set({ 6 "Books": 7 { 8 "Best_Sellers": -1 9 } 10 }) 11 12 ref = db.reference("/Books/Best_Sellers") 13 14 for key, value in data.items(): 15 ref.push().set(value)</pre> ✓ 2.0s The output will reset the previous data set in / node. The current data is shown below.



As you can see, under /Books/Best_Sellers there are 4 nodes where the node head (or node ID) is a randomly generated key which is due to the use of push() function. When data key does not matter, the use of push() function is desirable.

8

Update data:

Let's say the price of the books by J. R. R. Tolkien is reduced to 80 units to offer a discount. The first 3 books are written by this author, and we want to apply for a discount on all of them.

```
1 # Update data
2 #
3 # Requirement: The price of the books by
4 # J. R. R. Tolkien is reduced to 80 units to
5 # offer a discount.
6 #
7 ref = db.reference("/Books/Best_Sellers/")
8 best_sellers = ref.get()
9 print(best_sellers)
10 for key, value in best_sellers.items():
11     if(value["Author"] == "J.R.R. Tolkien"):
12         value["Price"] = 80
13         ref.child(key).update({"Price":80})
```

✓ 0.9s

As you can see, the author name is compared and the new price is set in the best_sellers dictionary and finally, an update() function is called on the ref, however, the current ref is a '/Books/Best_Sellers/', so we need to locate the

child under the ref node, so `ref.child(key)` is used in line 13. The output is shown below with a discounted price.



9

Delete data:

Let's delete all bestseller books with J.R.R. Tolkien as the author. You can locate the node using `db.reference()` (line 4) and then locate specific record (for loop in line 6) and calling `set()` with empty data `{}` as a parameter, such as `set({})`. The particular child under the ref needs to be located first by using `ref.child(key)`, otherwise, the ref node will be removed – **BE CAREFUL**.

```

1 # Let's delete all best seller books
2 # with J.R.R. Tolkien as the author.
3 #
4 ref = db.reference("/Books/Best_Sellers")
5
6 for key, value in best_sellers.items():
7     if(value["Author"] == "J.R.R. Tolkien"):
8         ref.child(key).set({})

```

This keeps only the other author data, as shown below.



If ref.child() not used, as shown the code below, all data will be removed.

```

1 ref = db.reference("/Books/Best_Sellers")
2 ref.set({})

```

Now in Firebase console you will see no data exists.

10 **Question:** Run all the cells in the Notebook you have downloaded in Step 4, fill in the student information at the top cell of the Notebook. Convert the Notebook to PDF and merge with this activity sheet PDF.

Answer: All cells in the provided firebase_explore.ipynb notebook were executed successfully. The student information was completed at the top of the notebook. The completed notebook was converted to PDF and will be merged with this activity sheet PDF.

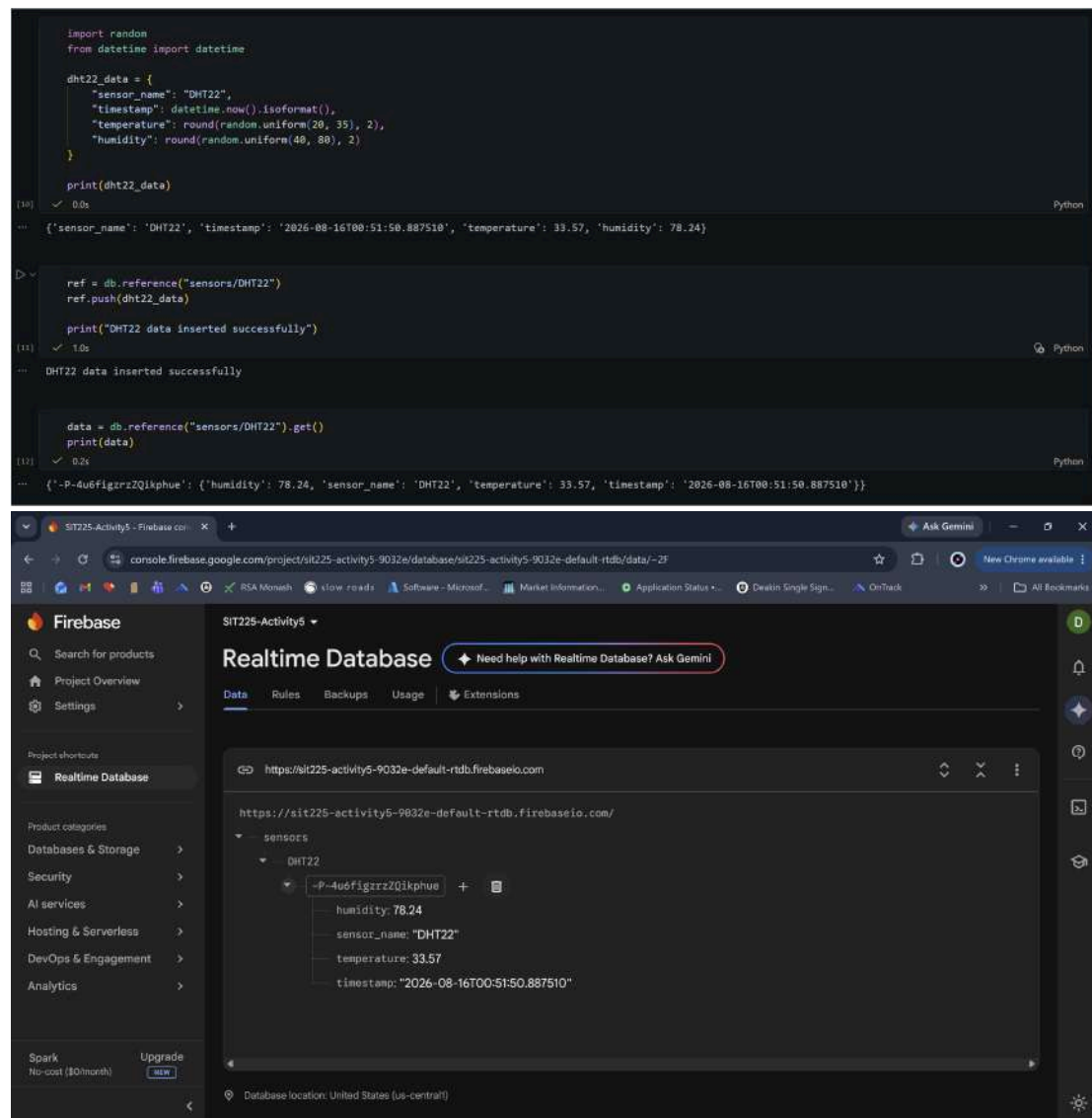
10 **Question:** Create a sensor data structure for DHT22 sensor which contains attributes such as sensor_name, timestamp, temperature and humidity. Remember there will be other sensors with different sensor variables such as DHT22 has 2 variables, accelerometer sensor has 3. For each such sensor, you will need to gather data over time. Discuss how you are going to handle multiple data values in JSON format? Justify your design.

Answer: I would store each sensor reading as a JSON object containing the sensor name, timestamp and its sensor measurements. For example, a DHT22 reading could contain sensor_name, timestamp, temperature and humidity. Multiple readings can be stored under unique keys, such as Firebase-generated keys using push(). This structure allows different sensors with different numbers of variables to be stored and collected over time without changing the overall database structure.

11

Question: Generate some random data for DHT22 sensor, insert data to database, query all data and screenshot the output here.

Answer: Random DHT22 sensor data containing sensor_name, timestamp, temperature and humidity was generated and inserted into the Firebase Realtime Database. The data was then queried successfully, and the output is shown in the screenshot below.



```
import random
from datetime import datetime

dht22_data = {
    "sensor_name": "DHT22",
    "timestamp": datetime.now().isoformat(),
    "temperature": round(random.uniform(20, 35), 2),
    "humidity": round(random.uniform(40, 80), 2)
}

print(dht22_data)

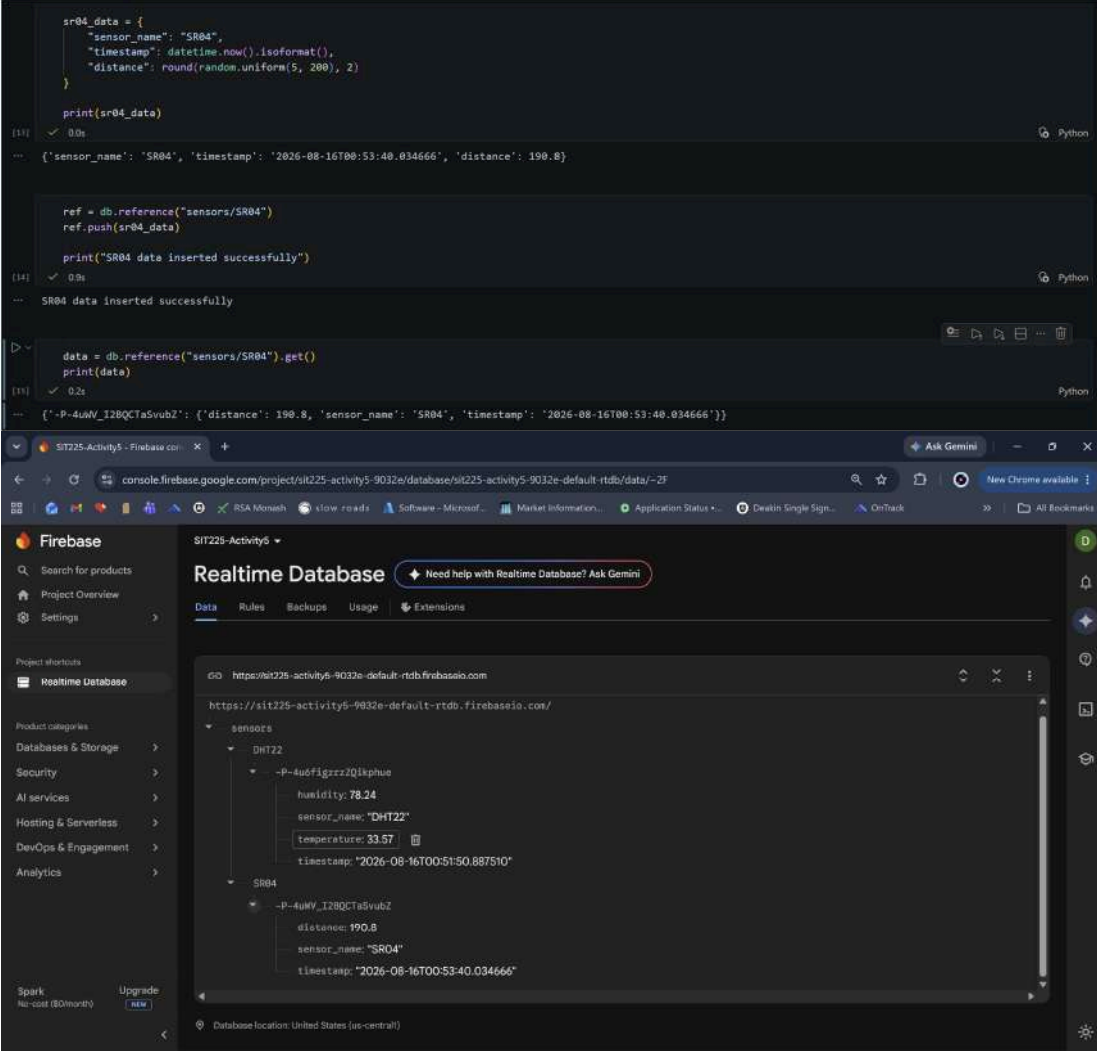
ref = db.reference("sensors/DHT22")
ref.push(dht22_data)

print("DHT22 data inserted successfully")

data = db.reference("sensors/DHT22").get()
print(data)
```

The screenshot of the Firebase console shows the Realtime Database structure. The data is stored under the path `sensors/DHT22` with a key `-P-4u6figrrz2Qikphue`. The data is a JSON object with the following fields:

- `humidity`: 78.24
- `sensor_name`: "DHT22"
- `temperature`: 33.57
- `timestamp`: "2026-08-16T00:51:50.887510"

12	Question: Generate some random data for the SR04 Ultrasonic sensor, insert data to database, query all data and screenshot the output here. Answer: Random SR04 ultrasonic sensor data containing sensor_name, timestamp and distance was generated and inserted into the Firebase Realtime Database. The stored records were then queried successfully, and the output is shown in the screenshot below.  <pre> sr04_data = { "sensor_name": "SR04", "timestamp": datetime.now().isoformat(), "distance": round(random.uniform(5, 200), 2) } print(sr04_data) [13]: ✓ 0.0s Python ... {'sensor_name': 'SR04', 'timestamp': '2026-08-16T00:53:40.034666', 'distance': 190.8} ref = db.reference("sensors/SR04") ref.push(sr04_data) print("SR04 data inserted successfully") [14]: ✓ 0.0s Python ... SR04 data inserted successfully data = db.reference("sensors/SR04").get() print(data) [15]: ✓ 0.2s Python ... {'-P-4uWV_I2BQCTa5vubZ': {'distance': 190.8, 'sensor_name': 'SR04', 'timestamp': '2026-08-16T00:53:40.034666'}} </pre> The screenshot of the Firebase Realtime Database console shows the following structure: <pre> sensors ├── DHT22 │ ├── -P-4u0f1grrz2Qikphue │ │ ├── humidity: 78.34 │ │ ├── sensor_name: "DHT22" │ │ ├── temperature: 33.57 │ │ └── timestamp: "2026-08-16T00:51:50.887510" │ └── SR04 │ ├── -P-4uWV_I2BQCTa5vubZ │ │ ├── distance: 190.8 │ │ ├── sensor_name: "SR04" │ │ └── timestamp: "2026-08-16T00:53:40.034666" </pre>
13	Question: Firebase Realtime database generates events on data operations. You can refer to section 'Handling Realtime Database events' in the document (https://firebase.google.com/docs/functions/database-events?gen=2nd). Discuss in the active learning session and summarise the idea of database events and how it is handled using Python SDK. Note that these events are useful when your sensors (from Arduino script) store data directly to Firebase Realtime database and you would like to track

	data update actions from a central Python application such as a monitoring dashboard. Answer: Firebase Realtime Database events are triggered when data in the database changes, such as when data is added, updated or deleted. These events allow an application to detect changes without repeatedly checking the database manually. Using the Python Firebase SDK, a Python application can listen for database changes and run a callback function when an event occurs. This is useful for a monitoring application because sensor data can be stored directly in Firebase while a central Python program responds to new or updated data in real time.
--	--

Activity 5.2: Data wrangling

Data wrangling is the process of converting raw data into a usable form. The process includes collecting, processing, analyzing, and tidying the raw data so that it can be easily read and analyzed. In this activity, you will use the common library in python, "pandas".

Hardware Required

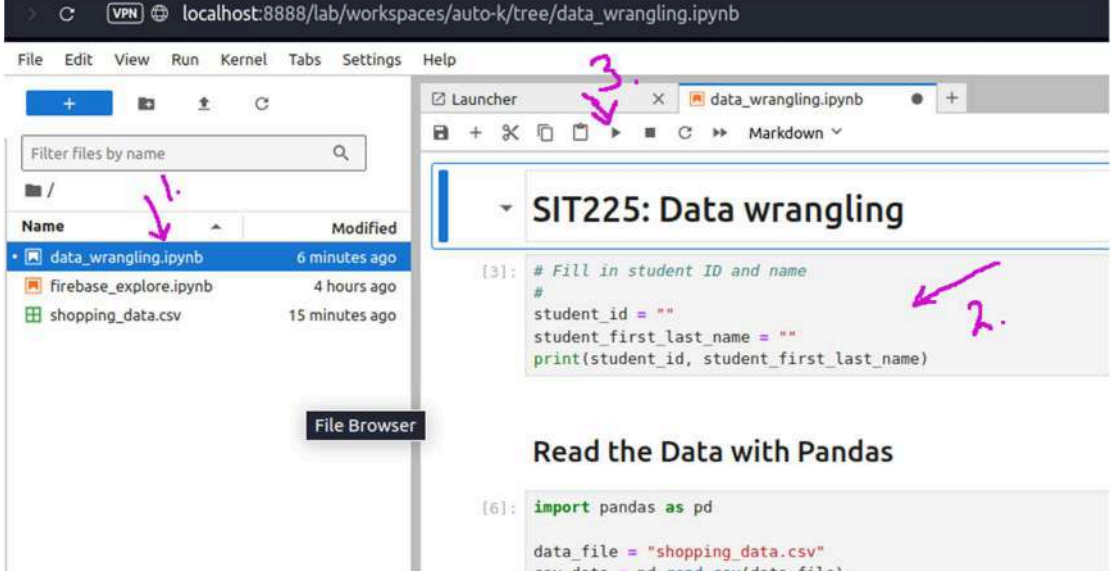
No hardware is required.

Software Required

Python 3
Pandas Python library

Steps

Step	Action
1	Install Pandas using the command below. Most likely you already have Pandas installed if you have installed Python using Anaconda distribution (https://www.anaconda.com/download). <pre>\$ pip install pandas</pre> A Python notebook is shared in the GitHub link (https://github.com/deakin-deep-dreamer/sit225/tree/main/week_5). There will be a data_wrangling.ipynb, shopping_data.csv and shopping_data_missingvalue.csv files among others. Download the week_5 folder in your computer, open a command prompt in that folder, and write the command below in the command line: <pre>\$ jupyter lab</pre> This will open Python Jupyter Notebook where in the left panel you can see the files (labeled as 1 in figure).

	 Each cell contains Python code (labeled as 2 in figure), you can run a cell by clicking on the cell, so the cursor appears in that cell and then click on the play button at the top of the panel (labeled as 3 in the figure).
2	Question: Run each cell to produce output. Follow instructions in the notebook to complete codes in some of the cells. Convert the notebook to PDF from menu File > Save and Export Notebook As > PDF. Convert this activity sheet to PDF and merge with the notebook PDF. Answer: There is no answer to write here. You have to answer in the Jupyter Notebook.
3	Question: Once you went through the cells in the Notebook, you now have a basic understanding of data wrangling. Pandas are a powerful tool and can be used for reading CSV data. Can you use Pandas in reading sensor CSV data that you generated earlier? Describe if any modification you think necessary? Answer: Yes, Pandas can be used to read sensor data stored in a CSV file. The CSV file would need to contain suitable column headings such as sensor name, timestamp, temperature, humidity or distance. Pandas can then load the data using read_csv() and provide functions for filtering, cleaning and analysing the sensor readings. Some modification may be required depending on the sensor, because different sensors produce different variables. For example, DHT22 data may contain temperature and humidity, while an ultrasonic sensor may contain distance. The CSV structure should therefore have separate columns for these measurements.

4

Question: What do you understand of the Notebook section called Handling Missing Value? Discuss in group and briefly summarise different missing value imputation methods and their applicability on different data conditions.

Answer: The Handling Missing Value section explains how missing or unavailable data can be identified and replaced so that it does not cause problems during analysis. Different methods can be used depending on the type of data. Missing values can be removed when only a small number of records are affected. For numerical sensor data, missing values can be replaced using the mean or median. The median is useful when the data contains outliers. For time-series sensor data, forward-fill or backward-fill can be useful because nearby readings are usually related. The most suitable method depends on the amount of missing data and the characteristics of the dataset.

```
In [1]: # Fill in student ID and name
#
student_id = "223861228"
student_first_last_name = "Deven Garg"
print(student_id, student_first_last_name)
```

223861228 Deven Garg

```
In [2]: """
        Firebase Realtime database demonstration.

        @Ahsan Habib
        School of IT,
        Deakin University, Australia.
        """

# Install Libraris, if not yet.
! pip install firebase_admin pandas

import firebase_admin

databaseURL = 'https://sit225-activity5-9032e-default-rtdb.firebaseio.com/'
cred_obj = firebase_admin.credentials.Certificate(
    'sit225-activity5-9032e-firebase-adminsdk-fbsvc-1d877f0713.json'
)
default_app = firebase_admin.initialize_app(cred_obj, {
    'databaseURL': databaseURL
})
```

Requirement already satisfied: firebase_admin in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (7.5.0)

Requirement already satisfied: pandas in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (3.0.3)

Requirement already satisfied: cachecontrol>=0.14.3 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from firebase_admin) (0.14.4)

Requirement already satisfied: google-api-core<3.0.0dev,>=2.25.1 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (2.34.0)

Requirement already satisfied: google-cloud-firestore>=2.21.0 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from firebase_admin) (2.28.1)

Requirement already satisfied: google-cloud-storage>=3.1.1 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from firebase_admin) (3.13.1)

Requirement already satisfied: pyjwt>=2.10.1 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from pyjwt[crypto]>=2.10.1->firebase_admin) (2.13.0)

Requirement already satisfied: httpx==0.28.1 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from httpx[http2]==0.28.1->firebase_admin) (0.28.1)

Requirement already satisfied: anyio in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (4.7.0)

Requirement already satisfied: certifi in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (2024.12.14)

Requirement already satisfied: httpcore==1.* in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (1.0.7)

Requirement already satisfied: idna in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (3.10)

Requirement already satisfied: h2<5,>=3 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from httpx[http2]==0.28.1->firebase_admin) (4.4.1)

Requirement already satisfied: googleapis-common-protos<2.0.0,>=1.69.2 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (1.75.1)

Requirement already satisfied: protobuf<8.0.0,>=6.33.5 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (7.35.0)

Requirement already satisfied: proto-plus<2.0.0,>=1.26.1 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (1.28.3)

Requirement already satisfied: google-auth<3.0.0,>=2.14.1 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (2.56.3)

Requirement already satisfied: requests<3.0.0,>=2.33.0 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (2.34.2)

Requirement already satisfied: grpcio<2.0.0,>=1.59.0 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (1.83.0)

Requirement already satisfied: grpcio-status<2.0.0,>=1.59.0 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (1.83.0)

Requirement already satisfied: pyasn1-modules>=0.2.1 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from google-auth<3.0.0,>=2.14.1->google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (0.4.2)

Requirement already satisfied: cryptography>=38.0.3 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages

s (from google-auth<3.0.0,>=2.14.1->google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (50.0.0)

Requirement already satisfied: typing-extensions~=4.12 in C:\Users\Deven Garg\AppData\Roaming\Python\Python312\site-packages (from grpcio<2.0.0,>=1.59.0->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (4.16.0)

Requirement already satisfied: hyperframe<7,>=6.1 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from h2<5,>=3->httpx[http2]==0.28.1->firebase_admin) (6.1.0)

Requirement already satisfied: hpack<5,>=4.2 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from h2<5,>=3->httpx[http2]==0.28.1->firebase_admin) (4.2.0)

Requirement already satisfied: h11<0.15,>=0.13 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from httpcore==1.*->httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (0.14.0)

Requirement already satisfied: charset_normalizer<4,>=2 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from requests<3.0.0,>=2.33.0->google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (3.4.0)

Requirement already satisfied: urllib3<3,>=1.26 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from requests<3.0.0,>=2.33.0->google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (2.2.3)

Requirement already satisfied: numpy>=1.26.0 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from pandas) (1.26.3)

Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: tzdata in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from pandas) (2026.2)

Requirement already satisfied: msgpack<2.0.0,>=0.5.2 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from cachecontrol>=0.14.3->firebase_admin) (1.2.1)

Requirement already satisfied: cffi>=2.0.0 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from cryptography>=38.0.3->google-auth<3.0.0,>=2.14.1->google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (2.0.0)

Requirement already satisfied: pycparser in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from cffi>=2.0.0->cryptography>=38.0.3->google-auth<3.0.0,>=2.14.1->google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (2.22)

Requirement already satisfied: google-cloud-core<3.0.0,>=2.0.0 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from google-cloud-firestore>=2.21.0->firebase_admin) (2.6.1)

Requirement already satisfied: google-resumable-media<3.0.0,>=2.7.2 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from google-cloud-storage>=3.1.1->firebase_admin) (2.10.1)

Requirement already satisfied: google-crc32c<2.0.0,>=1.6.0 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from google-cloud-storage>=3.1.1->firebase_admin) (1.8.0)

Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from pyasn1-modules>=0.2.1->google-auth<3.0.0,>=2.14.1->google-api-core<3.0.0dev,>=2.25.1->google-api-core[grpc]<3.0.0dev,>=2.25.1; platform_python_implementation != "PyPy"->firebase_admin) (0.6.4)

Requirement already satisfied: six>=1.5 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

Requirement already satisfied: sniffio>=1.1 in C:\Users\Deven Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages (from anyio->httpx==0.28.1->httpx[http2]==0.28.1->firebase_admin) (1.3.1)

In [3]: **from** firebase_admin **import** db

```
# A reference point is always needed to be set
# before any operation is carried out on a database.
```

```

#
ref = db.reference("/")

# JSON format data (key/value pair)
data = { # Outer {} contains inner data structure
    "Book1":
        {
            "Title": "The Fellowship of the Ring",
            "Author": "J.R.R. Tolkien",
            "Genre": "Epic fantasy",
            "Price": 100
        },
    "Book2":
        {
            "Title": "The Two Towers",
            "Author": "J.R.R. Tolkien",
            "Genre": "Epic fantasy",
            "Price": 100
        },
    "Book3":
        {
            "Title": "The Return of the King",
            "Author": "J.R.R. Tolkien",
            "Genre": "Epic fantasy",
            "Price": 100
        },
    "Book4":
        {
            "Title": "Brida",
            "Author": "Paulo Coelho",
            "Genre": "Fiction",
            "Price": 100
        }
    }

# JSON format data is set (overwritten) to the reference
# point set at /, which is the root node.
#
ref.set(data)

```

```

In [4]: ref = db.reference("/") # set ref point

```

```

# query all data under the ref
books = ref.get()
print(books)
print(type(books))

```

```

# print each item separately
for key, value in books.items():
    print(f"{key}: {value}")

# Query /Book1
ref = db.reference("/Book1")
books = ref.get()
print(books)

```

```

{'Book1': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}, 'Book2': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Two Towers'}, 'Book3': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Return of the King'}, 'Book4': {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'}}
<class 'dict'>
Book1: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}
Book2: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Two Towers'}
Book3: {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Return of the King'}
Book4: {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'}
{'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}

```

```

In [5]: # Write using push() function
# Note that a set() is called on top of push()
#
ref = db.reference("/")
ref.set({
    "Books":
    {
        "Best_Sellers": -1
    }
})

ref = db.reference("/Books/Best_Sellers")

for key, value in data.items():
    ref.push().set(value)

```

```

In [6]: # Update data
#
# Requirement: The price of the books by
# J. R. R. Tolkien is reduced to 80 units to
# offer a discount.
#
ref = db.reference("/Books/Best_Sellers/")
best_sellers = ref.get()
print(best_sellers)
for key, value in best_sellers.items():
    if(value["Author"] == "J.R.R. Tolkien"):

```

```
value["Price"] = 90
ref.child(key).update({"Price":80})
```

```
{'-P-4rY7C58nG0Pecg4oK': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Fellowship of the Ring'}, '-P-4rYESIOLZwtVyQMVR': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Two Towers'}, '-P-4rYLmdIhzaSU-OtpN': {'Author': 'J.R.R. Tolkien', 'Genre': 'Epic fantasy', 'Price': 100, 'Title': 'The Return of the King'}, '-P-4rYUdDccnxfk4cVXJ': {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'}}
```

```
In [7]: # Let's delete all best seller books
# with J.R.R. Tolkien as the author.
#
ref = db.reference("/Books/Best_Sellers")

for key, value in best_sellers.items():
    if(value["Author"] == "J.R.R. Tolkien"):
        ref.child(key).set({})
```

```
In [8]: # Delete all best_seller data.
#
ref = db.reference("/Books/Best_Sellers/")
best_sellers = ref.get()
print(best_sellers)
print(type(best_sellers))
```

```
{'-P-4rYUdDccnxfk4cVXJ': {'Author': 'Paulo Coelho', 'Genre': 'Fiction', 'Price': 100, 'Title': 'Brida'}}
<class 'dict'>
```

```
In [9]: ref = db.reference("/Books/Best_Sellers")
ref.set({})
```

SIT225: Data wrangling

Run each cell to generate output and finally convert this notebook to PDF.

```
In [1]: # Fill in student ID and name
#
student_id = "223861228"
student_first_last_name = "Deven Garg"
print(student_id, student_first_last_name)
```

223861228 Deven Garg

Read the Data with Pandas

Pandas has a dedicated function `read_csv()` to read CSV files.

Just in case we have a large number of data, we can just show into only five rows with head function. It will show you 5 rows data automatically.

```
In [2]: import pandas as pd

data_file = "shopping_data.csv"
csv_data = pd.read_csv(data_file)

print(csv_data)

# show into only five rows with head function
print(csv_data.head())
```

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40
..	...	...	...	...	...
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

[200 rows x 5 columns]

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Access the Column

Pandas has provided function `.columns` to access the column of the data source.

```
In [3]: print(csv_data.columns)

# if we want to access just one column, for example "Age"
print("Age:")
print(csv_data["Age"])
```

```
Index(['CustomerID', 'Genre', 'Age', 'Annual Income (k$)',  
      'Spending Score (1-100)'],  
      dtype='str')
```

Age:

```
0      19  
1      21  
2      20  
3      23  
4      31  
..  
195    35  
196    45  
197    32  
198    32  
199    30
```

Name: Age, Length: 200, dtype: int64

Access the Row

In addition to accessing data through columns, using pandas can also access using rows. In contrast to access through columns, the function to display data from a row is the `.iloc[i]` function where `[i]` indicates the order of the rows to be displayed where the index starts from 0.

In [4]: *# we want to know what line 5 contains*

```
print(csv_data.iloc[5])
```

```
print()
```

We can combine both of those function to show row and column we want.

For the example, we want to show the value in column "Age" at the first row

(remember that the row starts at 0)

#

```
print(csv_data["Age"].iloc[1])
```

```
CustomerID      6  
Genre           Female  
Age             22  
Annual Income (k$)  17  
Spending Score (1-100)  76  
Name: 5, dtype: object
```

21

Show Data Based on Range

After displaying a data set, what if you want to display data from rows 5 to 20 of a dataset? To anticipate this, pandas can also display data within a certain range, both ranges for rows only, only columns, and ranges for rows and columns

```
In [5]: print("Shows data to 5th to less than 10th in a row:")  
print(csv_data.iloc[5:10])
```

Shows data to 5th to less than 10th in a row:

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
5	6	Female	22	17	76
6	7	Female	35	18	6
7	8	Female	23	18	94
8	9	Male	64	19	3
9	10	Female	30	19	72

Using Numpy to Show the Statistic Information

The describe() function allows to quickly find statistical information from a dataset. Those information such as mean, median, modus, max min, even standard deviation. Don't forget to install Numpy before using describe function.

```
In [6]: print(csv_data.describe(include="all"))
```

	CustomerID	Genre	Age	Annual Income (k\$)	\
count	200.000000	200	200.000000	200.000000	
unique	NaN	2	NaN	NaN	
top	NaN	Female	NaN	NaN	
freq	NaN	112	NaN	NaN	
mean	100.500000	NaN	38.850000	60.560000	
std	57.879185	NaN	13.969007	26.264721	
min	1.000000	NaN	18.000000	15.000000	
25%	50.750000	NaN	28.750000	41.500000	
50%	100.500000	NaN	36.000000	61.500000	
75%	150.250000	NaN	49.000000	78.000000	
max	200.000000	NaN	70.000000	137.000000	

	Spending Score (1-100)
count	200.000000
unique	NaN
top	NaN
freq	NaN
mean	50.200000
std	25.823522
min	1.000000
25%	34.750000
50%	50.000000
75%	73.000000
max	99.000000

Handling Missing Value

```
In [7]: # For the first step, we will figure out if there is missing value.
print(csv_data.isnull().values.any())
print()
```

False

```
In [8]: # We will use another data source with missing values to practice this part.
data_missing = pd.read_csv("shopping_data_missingvalue.csv")
print(data_missing.head())

print()

print("Missing? ", data_missing.isnull().values.any())
```

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19.0	15.0	39.0
1	2	Male	NaN	15.0	81.0
2	3	Female	20.0	NaN	6.0
3	4	Female	23.0	16.0	77.0
4	5	Female	31.0	17.0	NaN

Missing? True

In []:

Ways to deal with missing values.

Follow the tutorial (<https://deeptime.com/app/rickyharyanto14-3390/Data-Wrangling-w-Python-e5d1a23e-33cf-416d-ad27-4c3f7f467442>). It includes -

1. Delete data

- deleting rows
- pairwise deletion
- delete column

2. imputation

- time series problem
 - Data without trend with seasonality (mean, median, mode, random)
 - Data with trend and without seasonality (linear interpolation)
- general problem
 - Data categorical (Make NA as multiple imputation)
 - Data numerical or continuous (mean, median, mode, multiple imputation and linear regression)

Filling with Mean Values

The mean is used for data that has a few outliers/noise/anomalies in the distribution of the data and its contents. This value will later fill in the empty value of the dataset that has a missing value case. To fill in an empty value use the `fillna()` function

In [9]: `print(data_missing.mean(numeric_only=True))`

"""

Question: This code will generate error. Can you explain why and how it can be solved?
Move on to the next cell to find one way it can be solved.

Answer: The original code produces an error because the dataset contains the Genre column,

which contains text values. The mean can only be calculated for numerical values.
Using numeric_only=True makes Pandas calculate the mean only for numerical columns.

```
"""
```

```
CustomerID      100.500000
Age             38.939698
Annual Income (k$) 61.005051
Spending Score (1-100) 50.489899
dtype: float64
```

Out[9]: '\n\nQuestion: This code will generate error. Can you explain why and how it can be solved? \nMove on to the next cell to find on e way it can be solved.\n\nAnswer: The original code produces an error because the dataset contains the Genre column, \nwhich con tains text values. The mean can only be calculated for numerical values. \nUsing numeric_only=True makes Pandas calculate the mea n only for numerical columns.\n\n'

```
In [10]: # Genre column contains string values and numeral operation mean fails.
# Lets drop Genre column since for numeral calculation.
#
data_missing_wo_genre = data_missing.drop(columns=['Genre'])
print(data_missing_wo_genre.head())
```

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	19.0	15.0	39.0
1	2	NaN	15.0	81.0
2	3	20.0	NaN	6.0
3	4	23.0	16.0	77.0
4	5	31.0	17.0	NaN

```
In [11]: print(data_missing_wo_genre.mean())
```

```
CustomerID      100.500000
Age             38.939698
Annual Income (k$) 61.005051
Spending Score (1-100) 50.489899
dtype: float64
```

```
In [12]: print("Dataset with empty values! :")
print(data_missing_wo_genre.head(10))

data_filling=data_missing_wo_genre.fillna(data_missing_wo_genre.mean())
print("Dataset that has been processed Handling Missing Values with Mean :")
print(data_filling.head(10))

# Observe the missing value imputation in corresponding rows.
#
```

Dataset with empty values! :

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	19.0	15.0	39.0
1	2	NaN	15.0	81.0
2	3	20.0	NaN	6.0
3	4	23.0	16.0	77.0
4	5	31.0	17.0	NaN
5	6	22.0	NaN	76.0
6	7	35.0	18.0	6.0
7	8	23.0	18.0	94.0
8	9	64.0	19.0	NaN
9	10	30.0	19.0	72.0

Dataset that has been processed Handling Missing Values with Mean :

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	19.000000	15.000000	39.000000
1	2	38.939698	15.000000	81.000000
2	3	20.000000	61.005051	6.000000
3	4	23.000000	16.000000	77.000000
4	5	31.000000	17.000000	50.489899
5	6	22.000000	61.005051	76.000000
6	7	35.000000	18.000000	6.000000
7	8	23.000000	18.000000	94.000000
8	9	64.000000	19.000000	50.489899
9	10	30.000000	19.000000	72.000000

Filling with Median

The median is used when the data presented has a high outlier. The median was chosen because it is the middle value, which means it is not the result of calculations involving outlier data. In some cases, outlier data is considered disturbing and often considered noisy because it can affect class distribution and interfere with clustering analysis.

```
In [13]: print(data_missing_wo_genre.median())
print("Dataset with empty values! :")
print(data_missing_wo_genre.head(10))

data_filling2=data_missing_wo_genre.fillna(data_missing_wo_genre.median())
print("Dataset that has been processed Handling Missing Values with Median :")
print(data_filling2.head(10))

# Observe the missing value imputation in corresponding rows.
#
```

```
CustomerID      100.5
Age              36.0
Annual Income (k$)  62.0
Spending Score (1-100)  50.0
dtype: float64
```

Dataset with empty values! :

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	19.0	15.0	39.0
1	2	NaN	15.0	81.0
2	3	20.0	NaN	6.0
3	4	23.0	16.0	77.0
4	5	31.0	17.0	NaN
5	6	22.0	NaN	76.0
6	7	35.0	18.0	6.0
7	8	23.0	18.0	94.0
8	9	64.0	19.0	NaN
9	10	30.0	19.0	72.0

Dataset that has been processed Handling Missing Values with Median :

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	19.0	15.0	39.0
1	2	36.0	15.0	81.0
2	3	20.0	62.0	6.0
3	4	23.0	16.0	77.0
4	5	31.0	17.0	50.0
5	6	22.0	62.0	76.0
6	7	35.0	18.0	6.0
7	8	23.0	18.0	94.0
8	9	64.0	19.0	50.0
9	10	30.0	19.0	72.0